LAPORAN PRAKTIKUM
DATA INGESTION – BATCH & API
MK PRAKTIKUM DATA PROCESSING WORKFLOW

Dosen Pengampu : Isbat Uzzin Nadhori S.Kom., M.T



Disusun Oleh:

Kadek Savita Dyutianaya

NRP. 3324600033

Sains Data Terapan B

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

PROGRAM STUDI SAINS DATA TERAPAN

2026

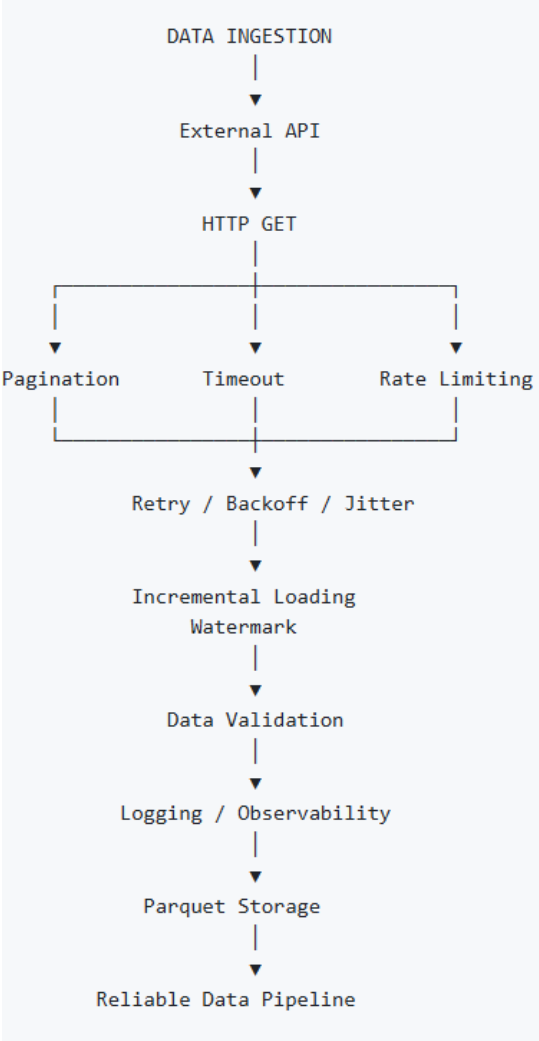
DAFTAR ISI

1. Tujuan Praktikum
2. Peta Kompetensi dan Keterkaitan dengan Materi Ajar
3. Hasil Akhir yang Harus Dicapai
4. Prasyarat
5. Persiapan Lingkungan
6. Struktur Project
7. Percobaan 0 — Menyiapkan Local Mock API
8. Percobaan 1 — Dasar Data Ingestion dan HTTP
9. Percobaan 2 — Pagination
10. Percobaan 3 — Rate Limiting dan HTTP 429
11. Percobaan 4 — Retry, Exponential Backoff, dan Jitter
12. Percobaan 5 — Incremental Loading dan Watermark
13. Percobaan 6 — Clean Code dan Modular Python
14. Percobaan 7 — Logging dan Observability
15. Percobaan 8 — Validasi Data
16. Percobaan 9 — Apache Parquet
17. Percobaan 10 — Pipeline End-to-End
18. Percobaan 11 — Strategi Penyimpanan Batch dan Partitioning

1. TUJUAN PRAKTIKUM

Setelah menyelesaikan modul ini, mahasiswa mampu membangun pipeline ingestion berbasis API

2. PETA PERCOBAAN

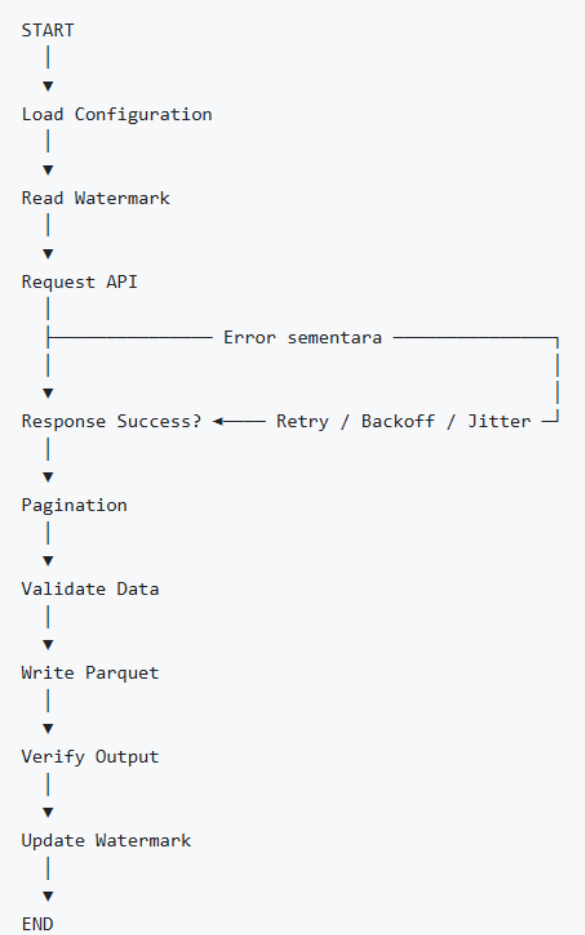


3. HASIL AKHIR PRAKTIKUM

Pada akhir praktikum, mahasiswa akan memiliki project seperti berikut:

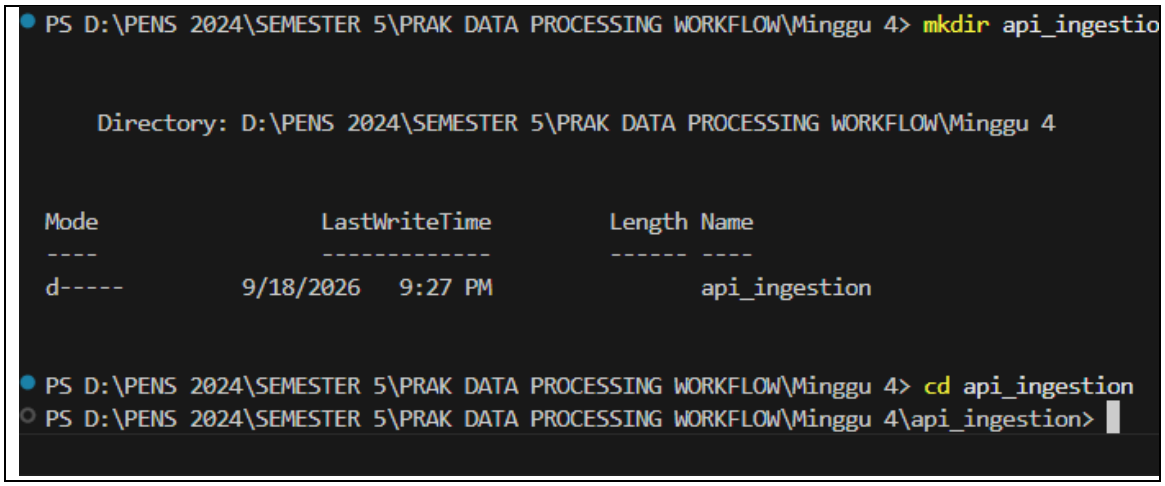
```
api_ingestion/
│
├── requirements.txt
├── mock_api.py
│
├── config.py
├── api_client.py
├── pagination.py
├── watermark.py
├── validator.py
├── storage.py
├── main.py
│
├── data/
│   ├── transactions/
│   │   └── transaction_date=YYYY-MM-DD/
│   │       └── transactions.parquet
│   └── ...
│
├── state/
│   └── watermark.json
│
└── logs/
    └── ingestion.log
```

Pipeline akhir:

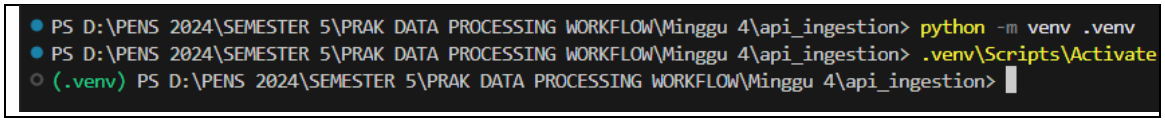


4. PERSIAPAN LINGKUNGAN

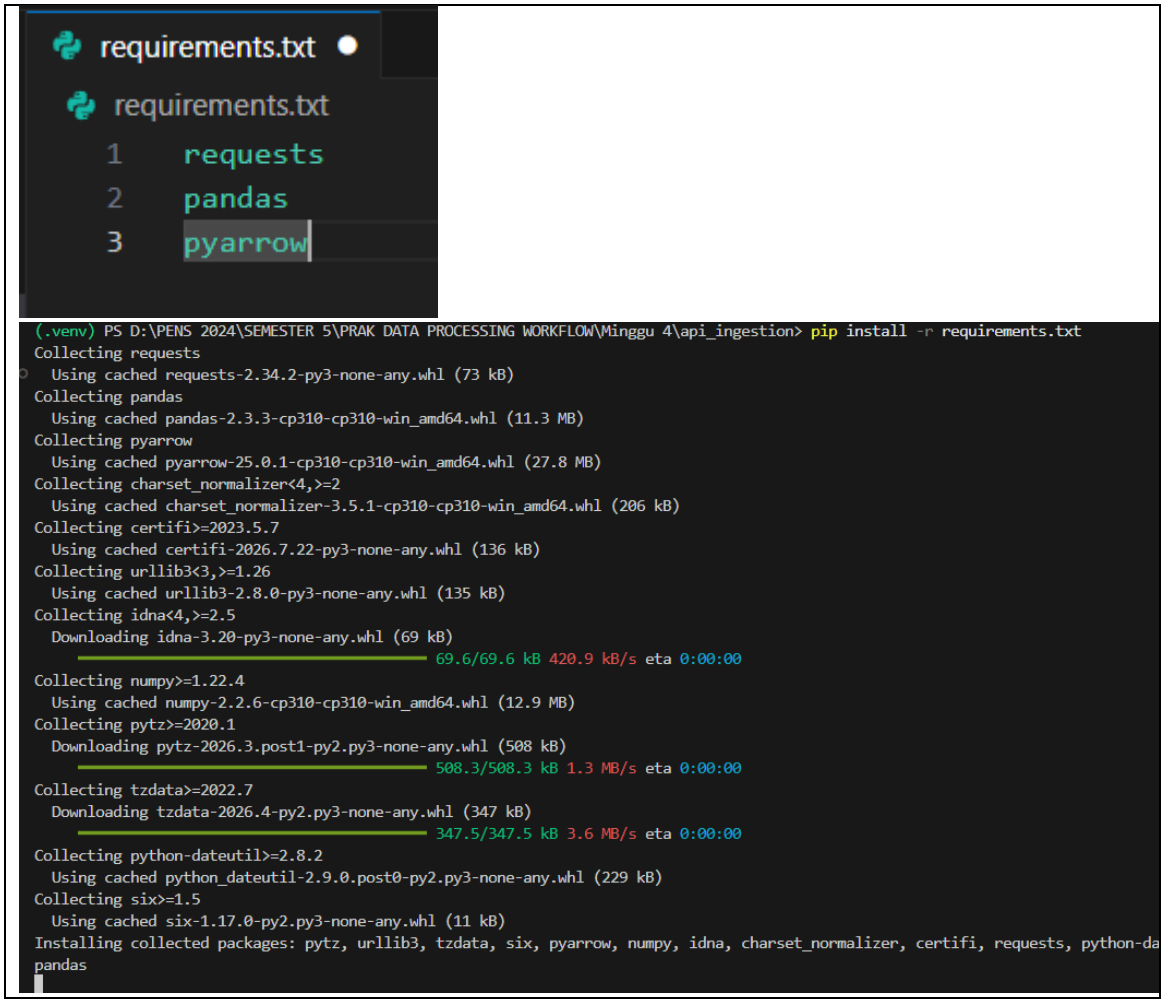
Langkah 1 — Buat folder project



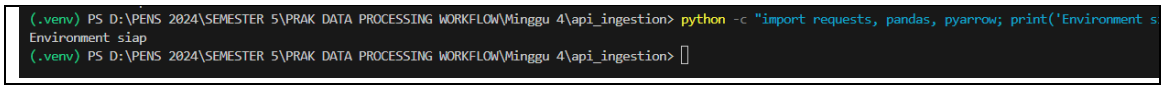
Langkah 2 — Buat virtual environment



Langkah 3 — Buat requirements.txt

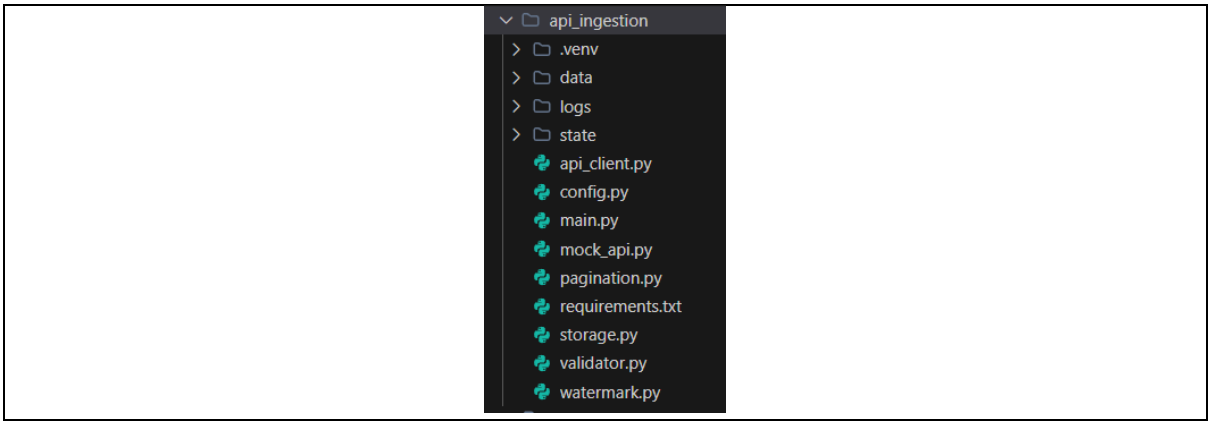


Langkah 4 — Verifikasi instalasi



5. STRUKTUR PROJECT AWAL

Buat struktur:



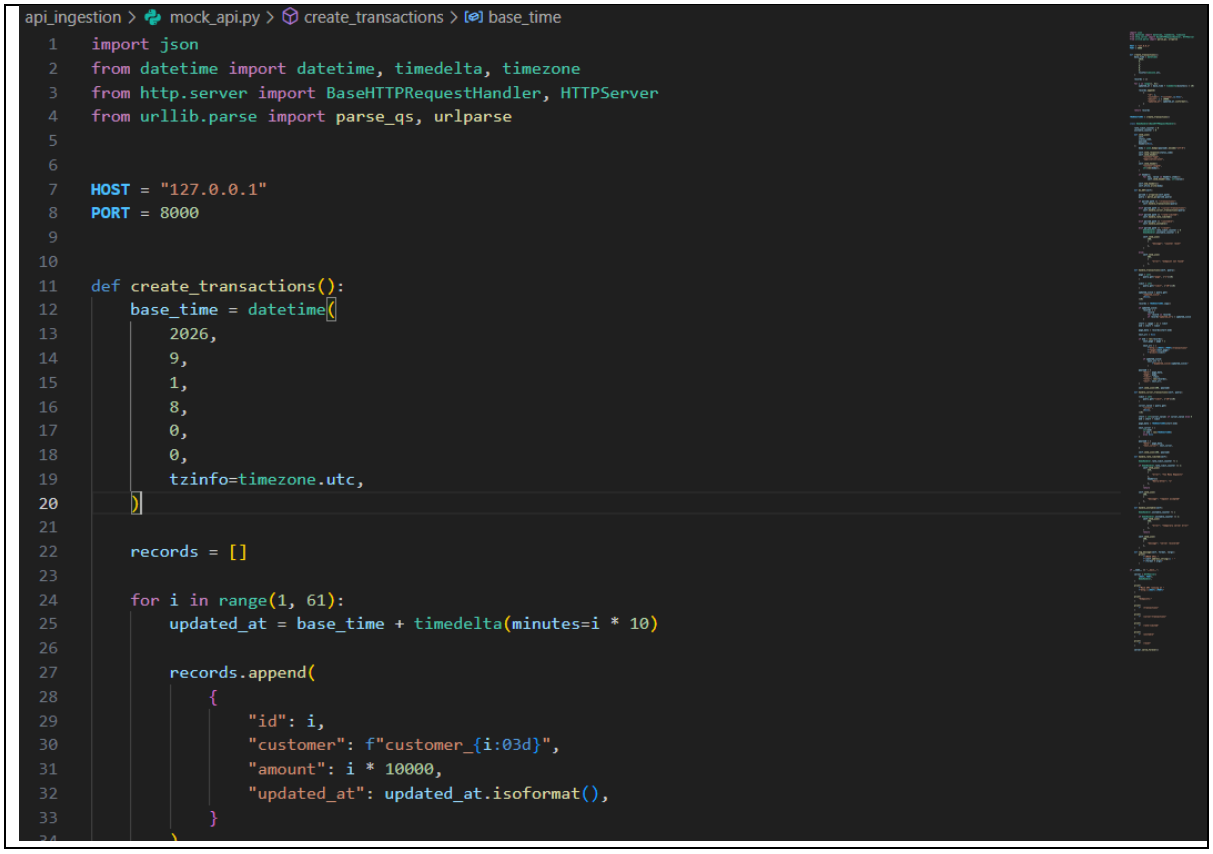
6. PERCOBAAN 0 — MENYIAPKAN LOCAL MOCK API

Tujuan

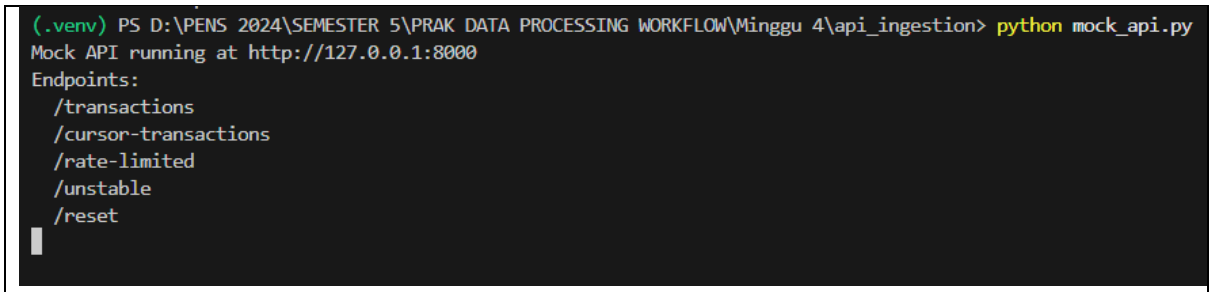
Sebelum mengakses API publik, kita membuat API lokal agar:

- Praktikum dapat dilakukan tanpa ketergantungan pada API pihak ketiga.
- Pagination dapat dikontrol.
- HTTP 429 dapat disimulasikan.
- Error 500 dapat disimulasikan.
- Watermark dapat diuji.

Langkah 1 — Buat mock_api.py



Langkah 2 — Jalankan API



After uji endpoint

```
(.venv) PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python mock_api.py
Mock API running at http://127.0.0.1:8000
Endpoints:
  /transactions
  /cursor-transactions
  /rate-limited
  /unstable
  /reset
[MOCK API] 127.0.0.1 - "GET /transactions HTTP/1.1" 200 -
[MOCK API] 127.0.0.1 - "GET /transactions?page=1&limit=5 HTTP/1.1" 200 -
[MOCK API] 127.0.0.1 - "GET /favicon.ico HTTP/1.1" 404 -
```

Langkah 3 — Uji endpoint

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4> python -c "import requests; print(requests.get('http://127.0.0.1
8000/transactions').json())"
{'data': [{'id': 1, 'customer': 'customer_001', 'amount': 10000, 'updated_at': '2026-09-01T08:10:00+00:00'}, {'id': 2, 'customer':
customer_002', 'amount': 20000, 'updated_at': '2026-09-01T08:20:00+00:00'}, {'id': 3, 'customer': 'customer_003', 'amount': 30000,
updated_at': '2026-09-01T08:30:00+00:00'}, {'id': 4, 'customer': 'customer_004', 'amount': 40000, 'updated_at': '2026-09-01T08:40:0
+00:00'}, {'id': 5, 'customer': 'customer_005', 'amount': 50000, 'updated_at': '2026-09-01T08:50:00+00:00'}, {'id': 6, 'customer':
customer_006', 'amount': 60000, 'updated_at': '2026-09-01T09:00:00+00:00'}, {'id': 7, 'customer': 'customer_007', 'amount': 70000,
updated_at': '2026-09-01T09:10:00+00:00'}, {'id': 8, 'customer': 'customer_008', 'amount': 80000, 'updated_at': '2026-09-01T09:20:0
+00:00'}, {'id': 9, 'customer': 'customer_009', 'amount': 90000, 'updated_at': '2026-09-01T09:30:00+00:00'}, {'id': 10, 'customer':
'customer_010', 'amount': 100000, 'updated_at': '2026-09-01T09:40:00+00:00'}], 'page': 1, 'limit': 10, 'total': 60, 'next': 'http:/
127.0.0.1:8000/transactions?page=2&limit=10'}
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4>
```

127.0.0.1:8000/transactions?page=1&limit=5

retty-print


```
"data": [
  {
    "id": 1,
    "customer": "customer_001",
    "amount": 10000,
    "updated_at": "2026-09-01T08:10:00+00:00"
  },
  {
    "id": 2,
    "customer": "customer_002",
    "amount": 20000,
    "updated_at": "2026-09-01T08:20:00+00:00"
  },
  {
    "id": 3,
    "customer": "customer_003",
    "amount": 30000,
    "updated_at": "2026-09-01T08:30:00+00:00"
  },
  {
    "id": 4,
    "customer": "customer_004",
    "amount": 40000,
    "updated_at": "2026-09-01T08:40:00+00:00"
  },
  {
    "id": 5,
    "customer": "customer_005",
    "amount": 50000,
    "updated_at": "2026-09-01T08:50:00+00:00"
  }
],
"page": 1,
"limit": 5,
"total": 60,
"next": "http://127.0.0.1:8000/transactions?page=2&limit=5"
```


7. PERCOBAAN 1 — DASAR DATA INGESTION DAN HTTP

Tujuan

Mahasiswa memahami:

- HTTP request.
- HTTP response.

- Status code.
- Headers.
- JSON body.
- Timeout.

Langkah 1 — Buat http_basic.py

api_ingestion > http_basic.py

```python
import requests


url = (
    "http://127.0.0.1:8000/"
    "transactions?page=1&limit=5"
)

response = requests.get(
    url,
    timeout=(3, 10),
)

print("Status Code:")
print(response.status_code)

print("\nHeaders:")
print(dict(response.headers))

print("\nJSON Body:")
print(response.json())
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python http_basic.py
Status Code:
200

Headers:
{'Server': 'BaseHTTP/0.6 Python/3.10.11', 'Date': 'Fri, 18 Sep 2026 15:04:14 GMT', 'Content-Type': 'application/json', 'Content-Le
th': '602'}

JSON Body:
{'data': [{'id': 1, 'customer': 'customer_001', 'amount': 10000, 'updated_at': '2026-09-01T08:10:00+00:00'}, {'id': 2, 'customer':
customer_002', 'amount': 20000, 'updated_at': '2026-09-01T08:20:00+00:00'}, {'id': 3, 'customer': 'customer_003', 'amount': 30000,
updated_at': '2026-09-01T08:30:00+00:00'}, {'id': 4, 'customer': 'customer_004', 'amount': 40000, 'updated_at': '2026-09-01T08:40:(
+00:00'}, {'id': 5, 'customer': 'customer_005', 'amount': 50000, 'updated_at': '2026-09-01T08:50:00+00:00'}], 'page': 1, 'limit': 5
 'total': 60, 'next': 'http://127.0.0.1:8000/transactions?page=2&limit=5'}
```

Langkah 2 — Analisis hasil

```
Status Code:
200
```

Status code 200 OK menandakan bahwa HTTP request berhasil diproses oleh server mock_api tanpa kendala. Server berhasil menerima query dan mengembalikan payload data yang diminta.

```
Headers:
{'Server': 'BaseHTTP/0.6 Python/3.10.11', 'Date': 'Fri, 18 Sep 2026 15:04:14 GMT', 'Content-Type': 'application/json', 'Content-Length':
'602'}
```

Header Content-Type memberitahu client (requests library di Python) bahwa format data yang dikirimkan oleh server dalam response body berupa objek JSON. Ini memungkinkan fungsi response.json() pada library requests untuk secara otomatis melakukan parsing string tersebut menjadi tipe data dict di Python

```
JSON Body:
{'data': [{'id': 1, 'customer': 'customer_001', 'amount': 10000, 'updated_at': '2026-09-01T08:10:00+00:00'}, {'id': 2, 'customer': 'custo
mer_002', 'amount': 20000, 'updated_at': '2026-09-01T08:20:00+00:00'}, {'id': 3, 'customer': 'customer_003', 'amount': 30000, 'updated_at
': '2026-09-01T08:30:00+00:00'}, {'id': 4, 'customer': 'customer_004', 'amount': 40000, 'updated_at': '2026-09-01T08:40:00+00:00'}, {'id'
: 5, 'customer': 'customer_005', 'amount': 50000, 'updated_at': '2026-09-01T08:50:00+00:00'}], 'page': 1, 'limit': 5, 'total': 60, 'next'
: 'http://127.0.0.1:8000/transactions?page=2&limit=5'}
```

- data (List of Objects): Berisi array record transaksi. Pada response ini terdapat 5 record transaksi pertama (ID 1 hingga 5), lengkap dengan atribut id, customer, amount, dan timestamp updated_at.
- page (Integer): Menunjukkan posisi halaman data saat ini, yaitu halaman 1.
- limit (Integer): Jumlah maksimum record yang dikembalikan dalam satu request/halaman, yaitu 5 record per halaman.
- total (Integer): Total keseluruhan record transaksi yang ada di server, yaitu 60 record.
- next (String / URL): URL endpoint untuk mengambil data halaman berikutnya (page=2). Properti ini adalah bagian penting dari strategi Pagination berbasis Next-Link.

Dari struktur body ini, dapat dihitung bahwa untuk mengambil seluruh 60 record data dengan limit=5, pipeline harus melakukan pengambilan data sebanyak 12 kali request 60/5 = 12 halaman), atau terus melakukan loop/fetch selama field "next" tidak bernilai null/None.

Langkah 3 — Tambahkan raise_for_status()

```
22
23    print("\nRaise for Status:")
24    print(response.raise_for_status())
25
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python http_basic.py
Status Code:
200

Headers:
{'Server': 'BaseHTTP/0.6 Python/3.10.11', 'Date': 'Fri, 18 Sep 2026 15:12:17 GMT', 'Content-Type': 'application/json', 'Content-Length':
'602'}

JSON Body:
{'data': [{'id': 1, 'customer': 'customer_001', 'amount': 10000, 'updated_at': '2026-09-01T08:10:00+00:00'}, {'id': 2, 'customer': 'custo
mer_002', 'amount': 20000, 'updated_at': '2026-09-01T08:20:00+00:00'}, {'id': 3, 'customer': 'customer_003', 'amount': 30000, 'updated_at
': '2026-09-01T08:30:00+00:00'}, {'id': 4, 'customer': 'customer_004', 'amount': 40000, 'updated_at': '2026-09-01T08:40:00+00:00'}, {'id'
: 5, 'customer': 'customer_005', 'amount': 50000, 'updated_at': '2026-09-01T08:50:00+00:00'}], 'page': 1, 'limit': 5, 'total': 60, 'next'
: 'http://127.0.0.1:8000/transactions?page=2&limit=5'}

Raise for Status:
None
```

```
Raise for Status:
None
```

Eksperimen

Ubah endpoint menjadi:

http://127.0.0.1:8000/not-found

```
url = (
    "http://127.0.0.1:8000/not-found"
)
```

Apa yang terjadi setelah:
response.raise_for_status()

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python http_basic.py
Status Code:
404

Headers:
{'Server': 'BaseHTTP/0.6 Python/3.10.11', 'Date': 'Fri, 18 Sep 2026 15:16:11 GMT', 'Content-Type': 'application/json', 'Content-Length
'31'}

JSON Body:
{'error': 'endpoint not found'}

Raise for Status:
Traceback (most recent call last):
  File "D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion\http_basic.py", line 23, in <module>
    print(response.raise_for_status())
  File "C:\Users\putuw\AppData\Local\Programs\Python\Python310\lib\site-packages\requests\models.py", line 1167, in raise_for_status
    raise HTTPError(http_error_msg, response=self)
requests.exceptions.HTTPError: 404 Client Error: Not Found for url: http://127.0.0.1:8000/not-found
```

- Status Code: Akan menampilkan 404 (Not Found).raise_for_status()
- Behavior: Memastikan bahwa response yang bukan 200 OK tidak lolos secara diam-diam ke tahap data processing. Jika status code >=400, fungsi ini langsung menghentikan alur program dengan melempar HTTPError.
- Penerapan di Pipeline: Di dalam api_client.py, raise_for_status() digunakan untuk mendeteksi kegagalan request sebelum data diproses lebih jauh atau disimpan ke database/storage.

Jawab Pertanyaan

a. Apa fungsi HTTP status code?

HTTP status code adalah kode respon standar 3-digit dari server yang berfungsi untuk memberitahukan status hasil pemrosesan HTTP request kepada client:

- 2xx (Success): Request berhasil diproses (misal 200 OK).
- 4xx (Client Error): Terjadi kesalahan dari sisi client (misal 404 Not Found, 429 Too Many Requests).
- 5xx (Server Error): Terjadi gangguan/kesalahan pada internal server (misal 500 Internal Server Error, 503 Service Unavailable)

b. Mengapa raise_for_status() berguna?
Fungsi raise_for_status() dari library requests sangat berguna untuk mencegah eror yang tersembunyi (silent failure). Secara default, library requests tidak melempar error meskipun server mengembalikan status error seperti 404 atau 500.

Dengan memanggil raise_for_status(), program akan secara otomatis melempar exception HTTPError jika status code bernilai >=400. Hal ini memastikan pipeline

data segera berhenti atau mengalihkan alur ke retry mechanism, bukannya lanjut memproses data kosong/salah yang dapat merusak kualitas data (data corruption).

c. Apa perbedaan connect timeout dan read timeout?
Dalam parameter timeout=(connect_timeout, read_timeout):
   - Connect Timeout: Batas waktu maksimum yang diberikan kepada client untuk membangun koneksi jaringan dengan server (melakukan TCP handshake). Jika server down atau unreachable, proses akan time out.
   - Read Timeout: Batas waktu maksimum yang ditunggu client untuk menerima byte data pertama/respon dari server setelah koneksi berhasil dibuat. Ini berguna jika server lambat atau menggantung (hang) saat memproses query/database.

d. Mengapa ingestion sebaiknya tidak menggunakan request tanpa timeout?
Jika request dipanggil tanpa parameter timeout (atau bernilai None), secara default Python requests akan menunggu tanpa batas waktu (indefinitely) hingga koneksi ditutup oleh server.

   Dalam data pipeline/ingestion, hal ini sangat berbahaya karena jika server API macet (hang) atau koneksi terputus tiba-tiba, proses ingestion akan membeku (freeze/stuck) selamanya. Akibatnya, sumber daya sistem terbuang, scheduled job (misal via Airflow/Cron) akan terus menggantung, dan pipeline berikutnya tidak akan pernah berjalan.

ANALISIS:
Percobaan ini melakukan eksplorasi dasar pengambilan data melalui protokol HTTP secara langsung. Penggunaan fitur timeout dan raise_for_status() diterapkan dengan tujuan mencegah program menggantung tanpa batas waktu apabila server tidak merespons. Selain itu, pengenalan terhadap HTTP status code sangat krusial dalam menentukan tindakan perbaikan selanjutnya secara otomatis jika terjadi kegagalan koneksi jaringan.

8. PERCOBAAN 2 — PAGINATION

8.1 Page Number Pagination

Tujuan

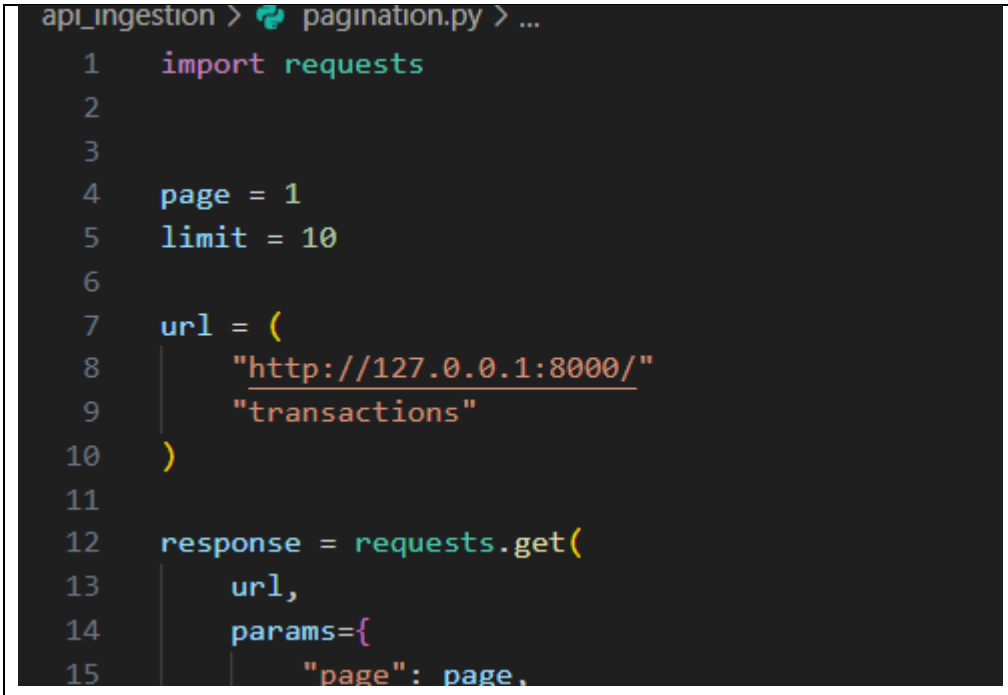
Mengambil seluruh data dari API yang membagi data menjadi beberapa halaman.

API:

```
GET /transactions?page=1&limit=10
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Ctrl click to launch VS Code Native REPL
>>> import requests
>>>
>>> # Menjalankan request GET ke endpoint transactions
>>> response = requests.get("http://127.0.0.1:8000/transactions?page=1&limit=10")
>>>
>>> print(response.status_code)
200
>>> print(response.json())
{'data': [{'id': 1, 'customer': 'customer_001', 'amount': 10000, 'updated_at': '2026-09-01T08:10:00+00:00'}, {'id': 2, 'customer': 'custo
mer_002', 'amount': 20000, 'updated_at': '2026-09-01T08:20:00+00:00'}, {'id': 3, 'customer': 'customer_003', 'amount': 30000, 'updated_at
': '2026-09-01T08:30:00+00:00'}, {'id': 4, 'customer': 'customer_004', 'amount': 40000, 'updated_at': '2026-09-01T08:40:00+00:00'}, {'id'
: 5, 'customer': 'customer_005', 'amount': 50000, 'updated_at': '2026-09-01T08:50:00+00:00'}, {'id': 6, 'customer': 'customer_006', 'amou
nt': 60000, 'updated_at': '2026-09-01T09:00:00+00:00'}, {'id': 7, 'customer': 'customer_007', 'amount': 70000, 'updated_at': '2026-09-01T
09:10:00+00:00'}, {'id': 8, 'customer': 'customer_008', 'amount': 80000, 'updated_at': '2026-09-01T09:20:00+00:00'}, {'id': 9, 'customer'
: 'customer_009', 'amount': 90000, 'updated_at': '2026-09-01T09:30:00+00:00'}, {'id': 10, 'customer': 'customer_010', 'amount': 100000, '
updated_at': '2026-09-01T09:40:00+00:00'}], 'page': 1, 'limit': 10, 'total': 60, 'next': 'http://127.0.0.1:8000/transactions?page=2&limit
=10'}
```

Langkah 1 — Ambil satu halaman




```
import requests


page = 1
limit = 10

url = (
    "http://127.0.0.1:8000/"
    "transactions"
)

response = requests.get(
    url,
    params={
```

```
        "page": page,
        "limit": limit,
    },
    timeout=(3, 10),
)

response.raise_for_status()

payload = response.json()

print(
    "Jumlah record:",
    len(payload["data"]),
)

for record in payload["data"]:
    print(record)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python pagination.py
Jumlah record: 10
{'id': 1, 'customer': 'customer_001', 'amount': 10000, 'updated_at': '2026-09-01T08:10:00+00:00'}
{'id': 2, 'customer': 'customer_002', 'amount': 20000, 'updated_at': '2026-09-01T08:20:00+00:00'}
{'id': 3, 'customer': 'customer_003', 'amount': 30000, 'updated_at': '2026-09-01T08:30:00+00:00'}
{'id': 4, 'customer': 'customer_004', 'amount': 40000, 'updated_at': '2026-09-01T08:40:00+00:00'}
{'id': 5, 'customer': 'customer_005', 'amount': 50000, 'updated_at': '2026-09-01T08:50:00+00:00'}
{'id': 6, 'customer': 'customer_006', 'amount': 60000, 'updated_at': '2026-09-01T09:00:00+00:00'}
{'id': 7, 'customer': 'customer_007', 'amount': 70000, 'updated_at': '2026-09-01T09:10:00+00:00'}
{'id': 8, 'customer': 'customer_008', 'amount': 80000, 'updated_at': '2026-09-01T09:20:00+00:00'}
{'id': 9, 'customer': 'customer_009', 'amount': 90000, 'updated_at': '2026-09-01T09:30:00+00:00'}
{'id': 10, 'customer': 'customer_010', 'amount': 100000, 'updated_at': '2026-09-01T09:40:00+00:00'}
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

Langkah 2 — Ambil semua halaman

```
import requests


url = (
    "http://127.0.0.1:8000/"
    "transactions"
)

page = 1
limit = 10

all_data = []

while True:

    response = requests.get(
```

```
        url,
        params={
            "page": page,
            "limit": limit,
        },
        timeout=(3, 10),
    )

    response.raise_for_status()

    payload = response.json()

    data = payload.get(
        "data",
        [],
    )

    if not data:
        break

    all_data.extend(data)

    print(
        f"Page {page}: "
        f"{len(data)} record"
    )

    page += 1


print(
    "Total record:",
    len(all_data),
)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python pagination.py
Page 1: 10 record
Page 2: 10 record
Page 3: 10 record
Page 4: 10 record
Page 5: 10 record
Page 6: 10 record
Total record: 60
```

8.2 Kesalahan Umum: Lupa Menaikkan Nomor Halaman

Kode salah:	Akibat:

```python
page = 1

while True:

    response = fetch_page(page)

    data = response["data"]

    if not data:
        break

    # page += 1 tidak ada
```

```
Page 1
Page 1
Page 1
Page 1
...
```

Ini adalah bentuk **infinite loop**.

Tugas

Tambahkan batas pengaman:
MAX_PAGES = 1000

```python
MAX_PAGES = 1000  # Batas pengaman
```

Kemudian:
if page > MAX_PAGES:
raise RuntimeError(
"Maximum page limit exceeded"
)

```python
while True:
    if page > MAX_PAGES:
        raise RuntimeError(f"Maximum page limit exceeded ({MAX_PAGES} pages)")
```

Jadi:

api_ingestion > pagination.py > ...

```python
import requests

url = "http://127.0.0.1:8000/transactions"
page = 1
limit = 10
MAX_PAGES = 1000  # Batas pengaman

all_data = []

while True:
    if page > MAX_PAGES:
        raise RuntimeError(f"Maximum page limit exceeded ({MAX_PAGES} pages)")

    response = requests.get(
        url,
        params={"page": page, "limit": limit},
        timeout=(3, 10),
    )
    response.raise_for_status()

    payload = response.json()
    data = payload.get("data", [])

    if not data:
        break

    all_data.extend(data)
    print(f"Page {page}: {len(data)} record")

    page += 1

print("Total record:", len(all_data))
```

8.3 Pagination dengan Generator

Pendekatan berikut menyimpan semua data:

```
all_data = []
all_data.extend(page_data)
```

Untuk data sangat besar, memory dapat meningkat.

Gunakan generator:



```python
import requests


def fetch_paginated_data(
    url,
    limit=10,
):

    page = 1

    while True:

        response = requests.get(
            url,
            params={
                "page": page,
                "limit": limit,
            },
            timeout=(3, 10),
        )

        response.raise_for_status()

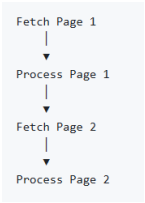
        payload = response.json()

        data = payload.get(
            "data",
            [],
        )
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python generator_test.py
Processing: 10 records
Processing: 10 records
Processing: 10 records
Processing: 10 records
Processing: 10 records
Processing: 10 records
```

Pada skrip generator ini, program tidak menyimpan seluruh data sekaligus di memori. Sebaliknya, saat bertemu kata kunci yield, fungsi akan "menyetorkan" 10 baris data ke perulangan di bawahnya, memprosesnya (dalam hal ini mem-print jumlahnya), lalu membuang memori 10 baris tadi untuk mengambil halaman berikutnya. Ini membuat pipeline kamu kebal dari masalah Out of Memory.

Konsep



Mahasiswa tidak harus menyimpan seluruh dataset sekaligus.

8.4 Next Link Pagination

API mock mengembalikan:

```
{
  "data": [],
  "next": "http://127.0.0.1:8000/transactions?page=2&limit=10"
}
```

Implementasi:

```python
import requests


url = (
    "http://127.0.0.1:8000/"
    "transactions?page=1&limit=10"
)

total = 0

while url:

    response = requests.get(
        url,
        timeout=(3, 10),
    )

    response.raise_for_status()

    payload = response.json()

    data = payload.get(
        "data",
        [],
    )

    total += len(data)

    print(
        "Fetched:",
        len(data),
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python next_link_test.py
Fetched: 10
Fetched: 10
Fetched: 10
Fetched: 10
Fetched: 10
Fetched: 10
Total: 60
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

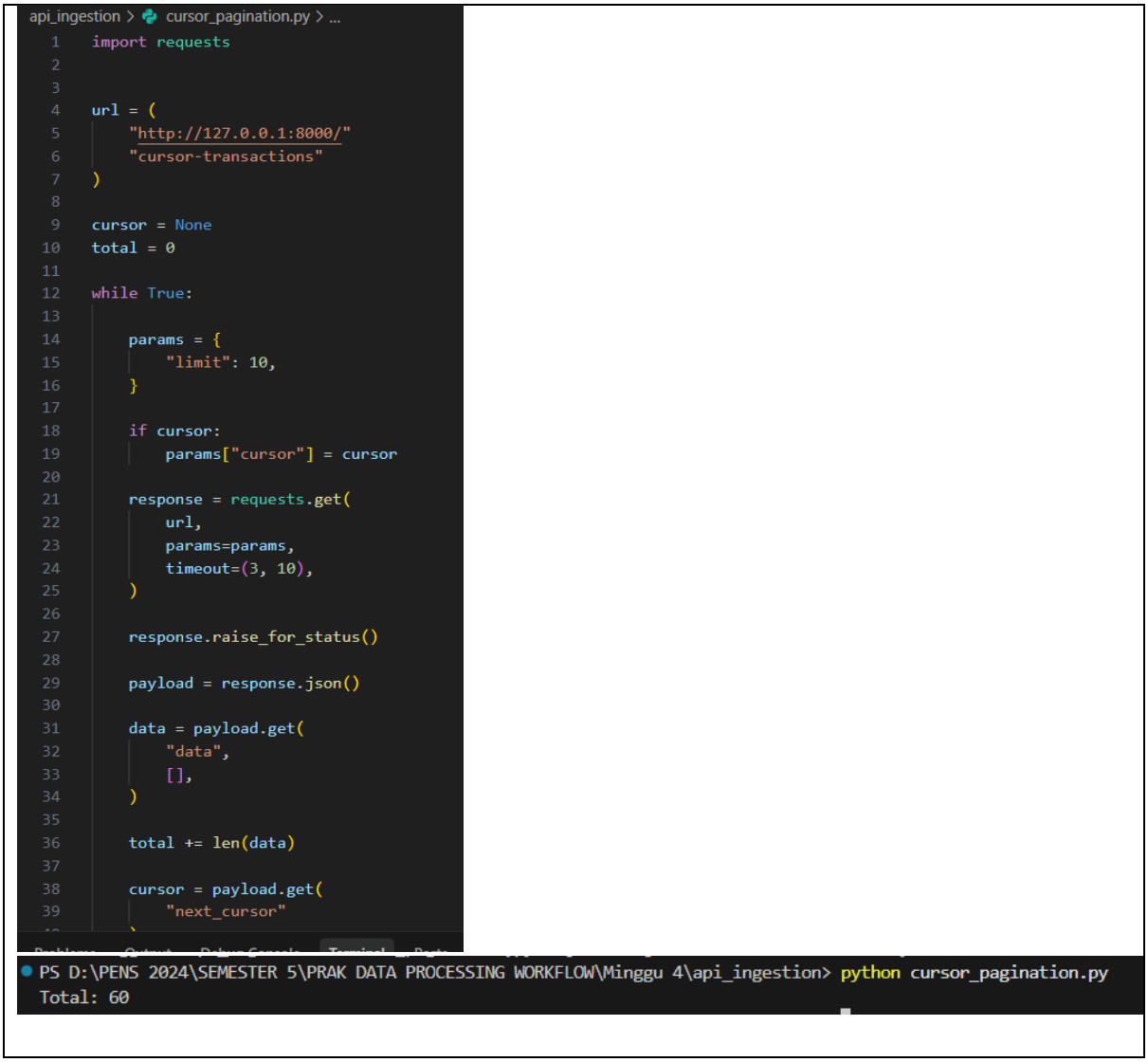
Prinsip:
Jika API menyediakan next, gunakan URL tersebut. Jangan mengasumsikan struktur parameter yang tidak dijamin oleh kontrak API.

8.5 Cursor Pagination

Endpoint:

/cursor-transactions?limit=10

Implementasi:



Prinsip penting

Cursor adalah **opaque token**.

Artinya:

```
Client menerima token
        ↓
Client menggunakan token
        ↓
Client tidak perlu mengetahui isi token
```

Jangan membuat asumsi seperti:

```
cursor = last_id + 10
```

kecuali API secara eksplisit mendefinisikan kontrak tersebut.

8.6 Tantangan Pagination

Buat fungsi:

```
def fetch_pages(
    url,
    limit=10,
):
    ...
```




```
import requests

def fetch_pages(url, limit=10):
    MAX_PAGES = 1000  # Syarat 5: Memiliki MAX_PAGES
    page = 1

    while True:
        # Pengecekan batas maksimal halaman
        if page > MAX_PAGES:
            raise RuntimeError(f"Melebihi batas MAX_PAGES ({MAX_PAGES})")

        # Syarat 2: Menggunakan timeout
        response = requests.get(
            url,
            params={"page": page, "limit": limit},
            timeout=(3, 10)
        )
```

```
        # Syarat 3: Memanggil raise_for_status()
        response.raise_for_status()

        payload = response.json()
        data = payload.get("data", [])

        # Syarat 4: Berhenti jika data kosong
        if not data:
            print("Data kosong, menghentikan pengambilan data.")
            break

        # Syarat 1: Menggunakan generator
        yield data

        page += 1

# --- BLOK UNTUK MENGUJI FUNGSI ---
if __name__ == "__main__":
    api_url = "http://127.0.0.1:8000/transactions"

    total_data = 0
    # Mengambil data menggunakan generator
    for batch in fetch_pages(api_url, limit=10):
        print(f"Berhasil mengambil {len(batch)} record data.")
        total_data += len(batch)

    print(f"\nTotal keseluruhan: {total_data} record.")
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python tantangan_pagination.py
Berhasil mengambil 10 record data.
Berhasil mengambil 10 record data.
Berhasil mengambil 10 record data.
Berhasil mengambil 10 record data.
Berhasil mengambil 10 record data.
Berhasil mengambil 10 record data.
Data kosong, menghentikan pengambilan data.

Total keseluruhan: 60 record.
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

Output di terminal membuktikan skrip berhasil melakukan iterasi sebanyak 6 kali untuk mengambil masing-masing 10 record, lalu secara otomatis berhenti saat mendeteksi data kosong, menghasilkan total akhir 60 record. Ini memastikan kelima syarat dari Tantangan 8.6 sudah terpenuhi dengan baik.

ANALISIS:

Praktikum ini menerapkan teknik pagination (pemanggilan data per halaman) untuk mengatasi beban memori saat mengambil data dalam jumlah yang sangat besar. Implementasi generator (yield) pada Python digunakan dengan tujuan agar sistem dapat memproses data secara bertahap tanpa harus menyimpannya sekaligus di dalam memori utama. Penambahan batas maksimal halaman

(MAX_PAGES) juga diujikan sebagai langkah mitigasi yang proaktif untuk melindungi program dari anomali perulangan tanpa henti (infinite loop).

9. PERCOBAAN 3 — RATE LIMITING DAN HTTP 429

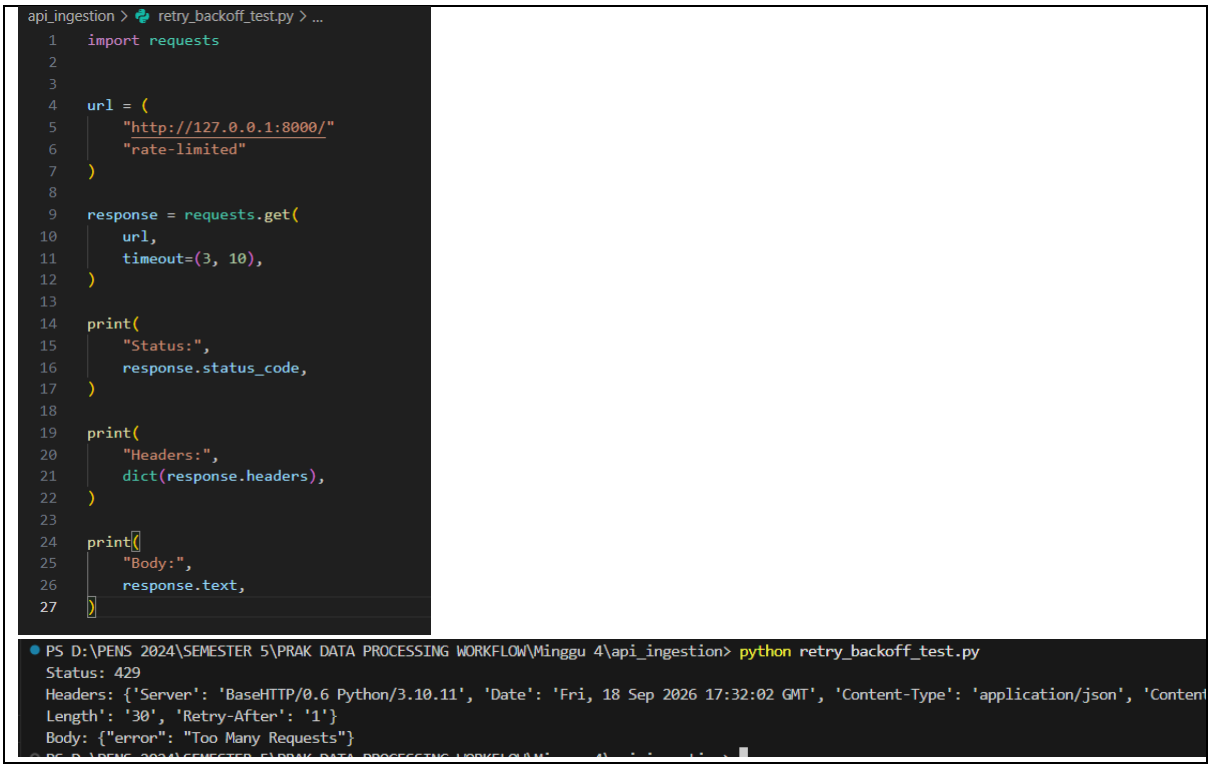
Tujuan

Memahami: HTTP 429 Too Many Requests

Endpoint: http://127.0.0.1:8000/rate-limited

Endpoint akan: Request 1 → 429 Request 2 → 429 Request 3 → 200 dan memberikan: Retry-After: 1

Langkah 1 — Uji tanpa penanganan



9.1 Pendekatan Kurang Baik

while True:

```
response = requests.get(url)

if response.status_code == 200:
    break
```

Masalah:

- Tidak ada timeout.
- Tidak ada batas retry.
- Dapat melakukan request terlalu cepat.
- Dapat memperparah beban server.
- Dapat menjadi infinite retry.

Modul

api_ingestion > watermark.py > save_watermark

```python
import json
import os

WATERMARK_FILE = "watermark.json"

def get_watermark():
    """
    Membaca watermark terakhir dari file JSON.
    Jika file belum ada (berarti ini load pertama kali), kembalikan tanggal default yang sangat lama.
    """
    if os.path.exists(WATERMARK_FILE):
        with open(WATERMARK_FILE, "r") as f:
            data = json.load(f)
            return data.get("last_updated_at")

    # Tanggal default tahun 1970 untuk memastikan semua data tertarik saat Full Load pertama
    return "1970-01-01T00:00:00+00:00"

def save_watermark(last_updated_at):
    """
    Menyimpan watermark (timestamp) terbaru ke file JSON.
    Hanya dipanggil SETELAH data benar-benar berhasil disimpan ke storage.
    """
    with open(WATERMARK_FILE, "w") as f:
        json.dump({"last_updated_at": last_updated_at}, f, indent=4)

    print(f"[Watermark] Disimpan: {last_updated_at}")
```

Skrip pengujian

api_ingestion > watermark_test.py > ...

```python
from watermark import get_watermark, save_watermark

print("=== UJI COBA WATERMARK / INCREMENTAL LOAD ===")

# 1. Baca watermark awal (pasti 1970 karena file belum ada)
current_wm = get_watermark()
print(f"1. Watermark awal: {current_wm}")

# 2. Simulasi pipeline mengambil data baru
print("2. Simulasi memproses data baru dari API...")
data_baru = [
    {"id": 1, "updated_at": "2026-09-18T08:10:00+00:00"},
    {"id": 2, "updated_at": "2026-09-18T08:20:00+00:00"},
    {"id": 3, "updated_at": "2026-09-19T09:00:00+00:00"}  # Ini yang paling baru
]

# 3. Cari timestamp terbesar/terbaru dari batch data tersebut
latest_wm = current_wm
for record in data_baru:
    if record["updated_at"] > latest_wm:
        latest_wm = record["updated_at"]

# 4. Simpan watermark terbaru setelah data "berhasil" diproses
print("3. Menyimpan watermark terbaru...")
save_watermark(latest_wm)

# 5. Baca ulang untuk verifikasi
print(f"4. Watermark saat ini di sistem: {get_watermark()}")
```

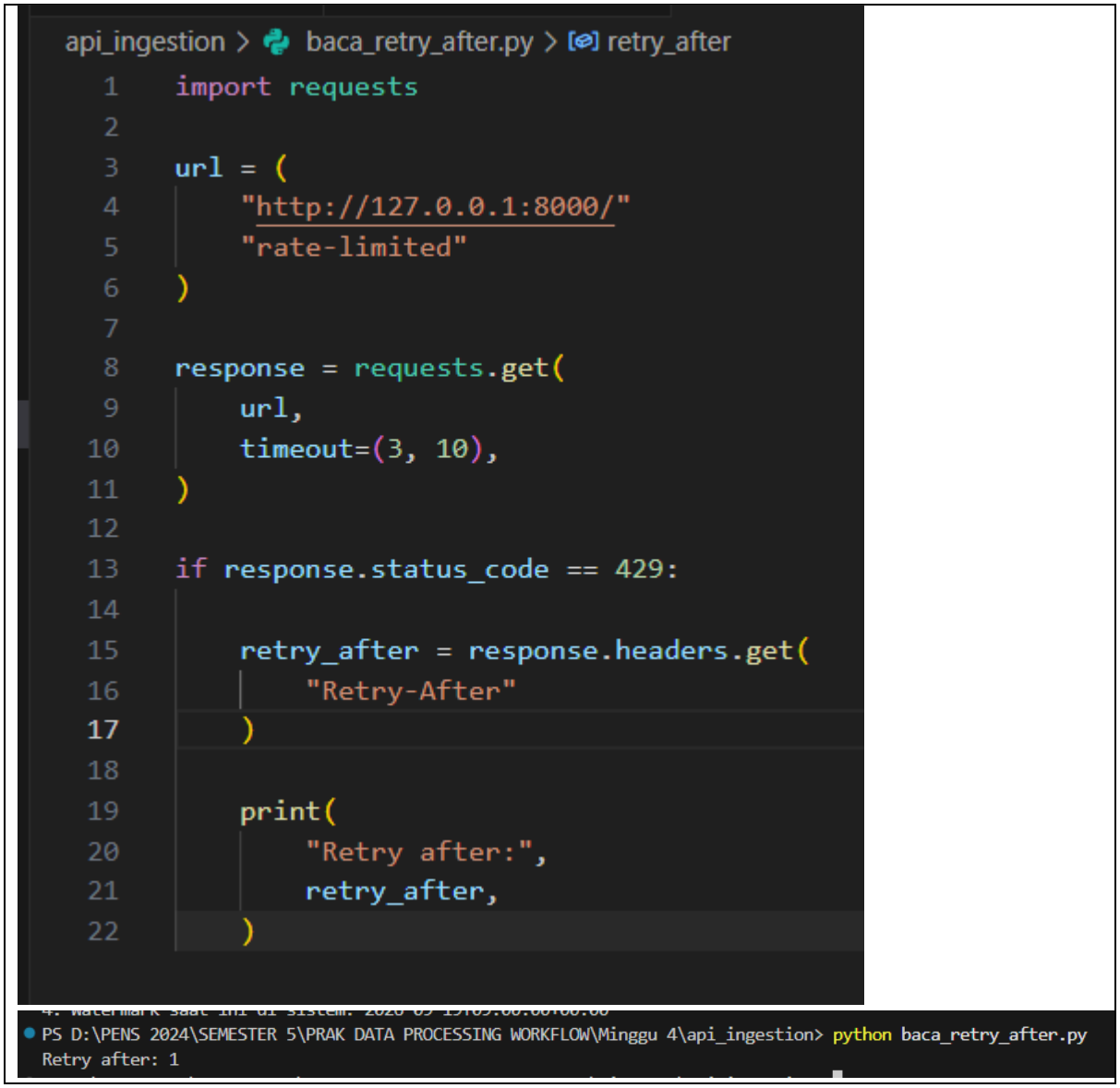
Hasil

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python watermark_test.py
=== UJI COBA WATERMARK / INCREMENTAL LOAD ===
1. Watermark awal: 1970-01-01T00:00:00+00:00
2. Simulasi memproses data baru dari API...
3. Menyimpan watermark terbaru...
[Watermark] Disimpan: 2026-09-19T09:00:00+00:00
4. Watermark saat ini di sistem: 2026-09-19T09:00:00+00:00
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

Saat pertama kali dijalankan, sistem membaca watermark awal dari tahun 1970 karena file watermark.json belum ada. Ini memastikan semua data dari awal waktu akan ditarik.

Program kemudian memproses simulasi data dan berhasil mendeteksi timestamp terbaru, yaitu 2026-09-19T09:00:00+00:00. Watermark baru tersebut berhasil disimpan dan terbaca kembali oleh sistem.

10. Membaca Header Retry-After



10.1 Penanganan 429

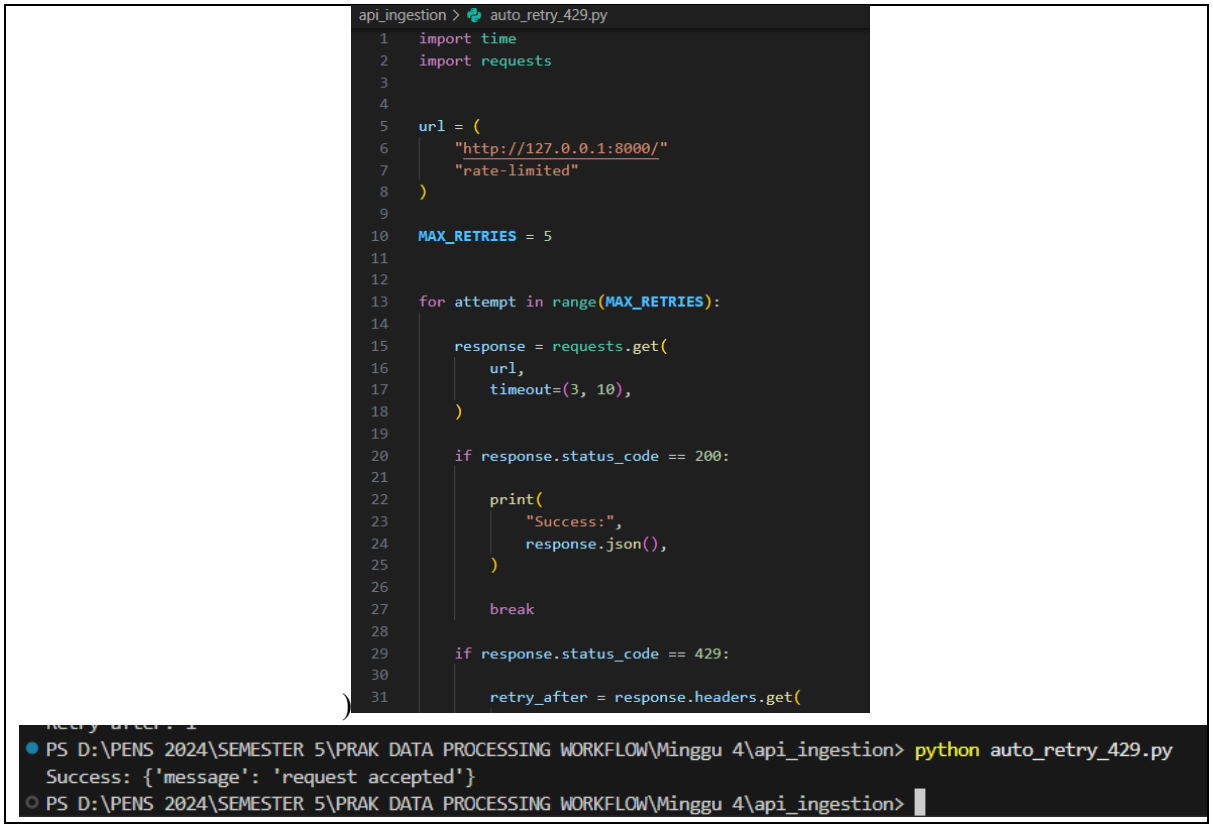
Sebelum menguji ulang, reset API: http://127.0.0.1:8000/reset

Atau:

import requests

requests.get(

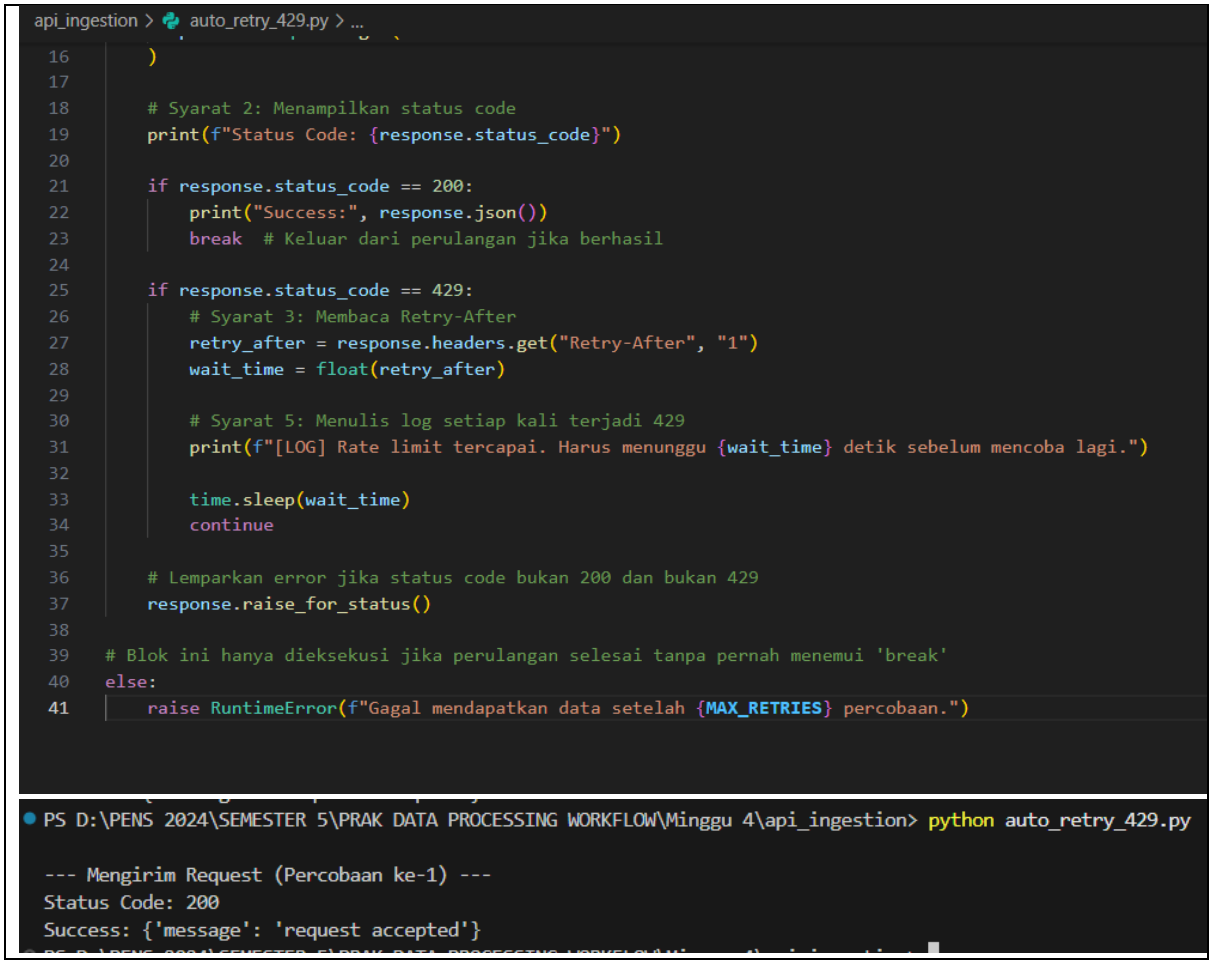
"http://127.0.0.1:8000/reset"



Tugas

Modifikasi kode agar:

1. Menampilkan nomor percobaan.
2. Menampilkan status code.
3. Membaca Retry-After.
4. Berhenti setelah maksimal 5 percobaan.
5. Menulis log setiap kali terjadi 429



ANALISIS:

Percobaan ini menyimulasikan cara merespons batasan rate limit dari server API yang diidentifikasi melalui kemunculan error HTTP 429. Logika pembacaan header Retry-After diimplementasikan dengan tujuan menjeda eksekusi program sesuai dengan instruksi waktu tunggu dari sisi server. Pendekatan ini memastikan bahwa rancangan program client mematuhi kapasitas server agar tidak memperparah beban sistem aplikasi secara keseluruhan.

11. PERCOBAAN 4 — RETRY, EXPONENTIAL BACKOFF, DAN JITTER

Tujuan

Membangun retry yang:

- Tidak dilakukan untuk semua error.
- Memberi waktu server untuk pulih.
- Tidak membuat banyak client melakukan retry secara serempak.

11.1 Simulasi Error Sementara

Endpoint: http://127.0.0.1:8000/unstable Perilaku: - Request 1 → 500 - Request 2 → 500

- Request 3 → 200

Reset:

```
import requests
requests.get(
   "http://127.0.0.1:8000/reset")
```

api_ingestion > test_unstable.py > ...

```python
import requests

# 1. Reset kondisi server agar hitungannya kembali dari awal
print("--- Mereset Server ---")
requests.get("http://127.0.0.1:8000/reset")
print("Server di-reset.")

# 2. Menembak endpoint unstable sebanyak 3 kali berturut-turut
url_unstable = "http://127.0.0.1:8000/unstable"

for i in range(1, 4):
    print(f"\nMengirim Request ke-{i}...")
    response = requests.get(url_unstable)
    print(f"Status Code: {response.status_code}")
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python test_unstable.py
--- Mereset Server ---
Server di-reset.

Mengirim Request ke-1...
Status Code: 500

Mengirim Request ke-2...
Status Code: 500

Mengirim Request ke-3...
Status Code: 200
```

11.2 Retry Manual

api_ingestion > retry_manual.py > ...

```python
import time
import requests


url = (
    "http://127.0.0.1:8000/"
    "unstable"
)

MAX_RETRIES = 5


for attempt in range(
    1,
    MAX_RETRIES + 1,
):

    response = requests.get(
        url,
        timeout=(3, 10),
    )

    print(
        f"Attempt {attempt}: "
        f"status={response.status_code}"
    )

    if response.status_code == 200:

        print(
            "Success:",
            response.json(),
        )

        break
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python retry_manual.py
Attempt 1: status=500
Attempt 2: status=500
Attempt 1: status=500
Attempt 2: status=500
Attempt 2: status=500
Attempt 3: status=200
Success: {'message': 'server recovered'}
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

11.3 Mengapa Retry Tidak Boleh Tanpa Batas?

Kode salah:

```
while True:

    try:
        response = requests.get(url)
        response.raise_for_status()
        break

    except Exception:
        continue
```

Masalah:

```
ERROR
  ↓
Retry
  ↓
ERROR
  ↓
Retry
  ↓
ERROR
  ↓
...
```

Akibat:

Server terus dibebani.

Program tidak selesai.

Error permanen tidak ditangani dengan benar.

Biaya komputasi meningkat.

Sulit melakukan observasi.

11.4 Exponential Backoff

wait = base_delay × 2^(attempt - 1)

jika**:**
base_delay = 1

maka;

Attempt	Wait
1	1 detik
2	2 detik
3	4 detik
4	8 detik

implementasi:

```
base_delay = 1

for attempt in range(1, 5):

    wait_time = (
        base_delay
        * (2 ** (attempt - 1))
```

```
    )

    print(
        f"Attempt {attempt}: "
        f"wait {wait_time}"
    )
```

11.5 Retry + Exponential Backoff

```
(venv) PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW> python "d:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4
pi_ingestion\retry_exponential_backoff.py"
Temporary error. Wait 1 seconds
Temporary error. Wait 2 seconds
Success: {'message': 'server recovered'}
```

11.6 Jitter

- Masalah tanpa jitter

Bayangkan 100 client:

```
10:00:00 → ERROR
10:00:01 → RETRY bersama
```

Semua melakuka retry bersamaan

- Jitter

Tambahkan variasi:

Minggu 4 > api_ingestion > retry_exponential_backoff.py > wait_time

```
35          502,
36          503,
37          504,
38      ):
39
40          if attempt == MAX_RETRIES:
41              break
42
43          import random
44
45
46          wait_time = (
47              BASE_DELAY
48              * (2 ** (attempt - 1))
49          )
50
51          jitter = random.uniform(
52              0,
53              1,
54          )
55
56          sleep_time = (
57              wait_time + jitter
58          )
```

```
(venv) PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW> & C:/Users/putuw/AppData/Local/Programs/Python/Python312/python.exe "d:
PENS 2024/SEMESTER 5/PRAK DATA PROCESSING WORKFLOW/Minggu 4/api_ingestion/retry_exponential_backoff.py"
Success: {'message': 'server recovered'}
```

11.7 Retry + Backoff + Jitter



11.8 Error Sementara vs Error yang Perlu Pemeriksaan

Contoh yang dapat dipertimbangkan untuk retry:

```
Timeout
Connection error
429
500
502
503
504
```

Contoh yang biasanya tidak boleh di-retry secara buta:

```
400
401
403
404
```

Alasan:

```
401 → kemungkinan kredensial salah
403 → kemungkinan permission salah
404 → kemungkinan endpoint salah
400 → kemungkinan request salah
```

Retry tidak akan memperbaiki request yang salah.

11.9 Tugas

Tambahkan:

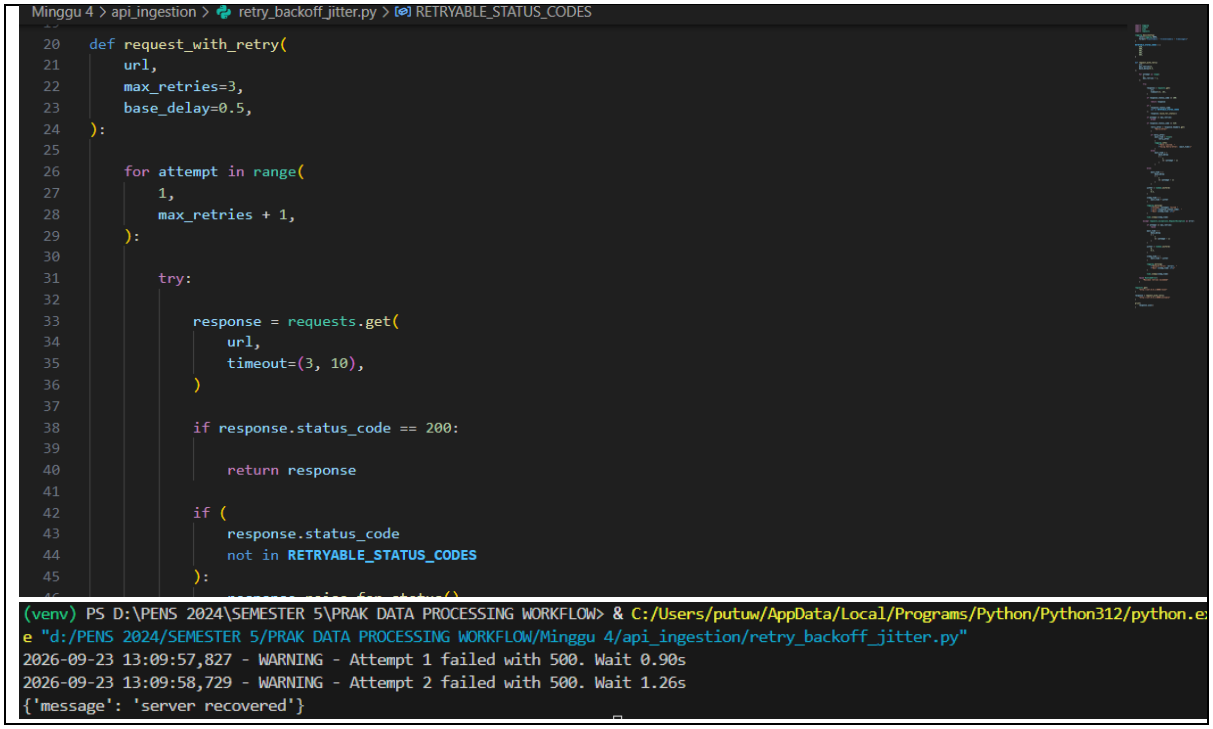
MAX_RETRIES = 3

BASE_DELAY = 0.5

Jitter maksimum 0.5 detik.

Logging untuk setiap retry.

Penanganan 429 yang memprioritaskan Retry-After.



ANALISIS:

Modul ini mempraktikkan penerapan strategi Exponential Backoff dan Jitter dengan tujuan menangani error yang bersifat sementara pada server (seperti kode HTTP 500, 502, 503, dan 504). Penambahan nilai Jitter, berupa penundaan waktu yang diacak, terbukti sangat krusial untuk mencegah terjadinya lonjakan antrean retry yang serentak dari banyak client pada saat yang bersamaan. Perpaduan algoritma ini menghasilkan sistem penarikan data yang jauh lebih toleran terhadap kegagalan dan aman bagi stabilitas infrastruktur API.

12. PERCOBAAN 5 — INCREMENTAL LOADING DAN WATERMARK

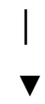
Tujuan

Menghindari: Full Load ↓ Mengambil seluruh data ↓ Setiap kali program berjalan
Kita ingin: Run 1 ↓ Ambil data baru Run 2 ↓

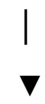
Ambil data setelah checkpoint sebelumnya

12.1 Full Load

API



Record 1–1.000.000



Pipeline

12.2 Incremental Loading

Gunakan watermark:

Last successful watermark

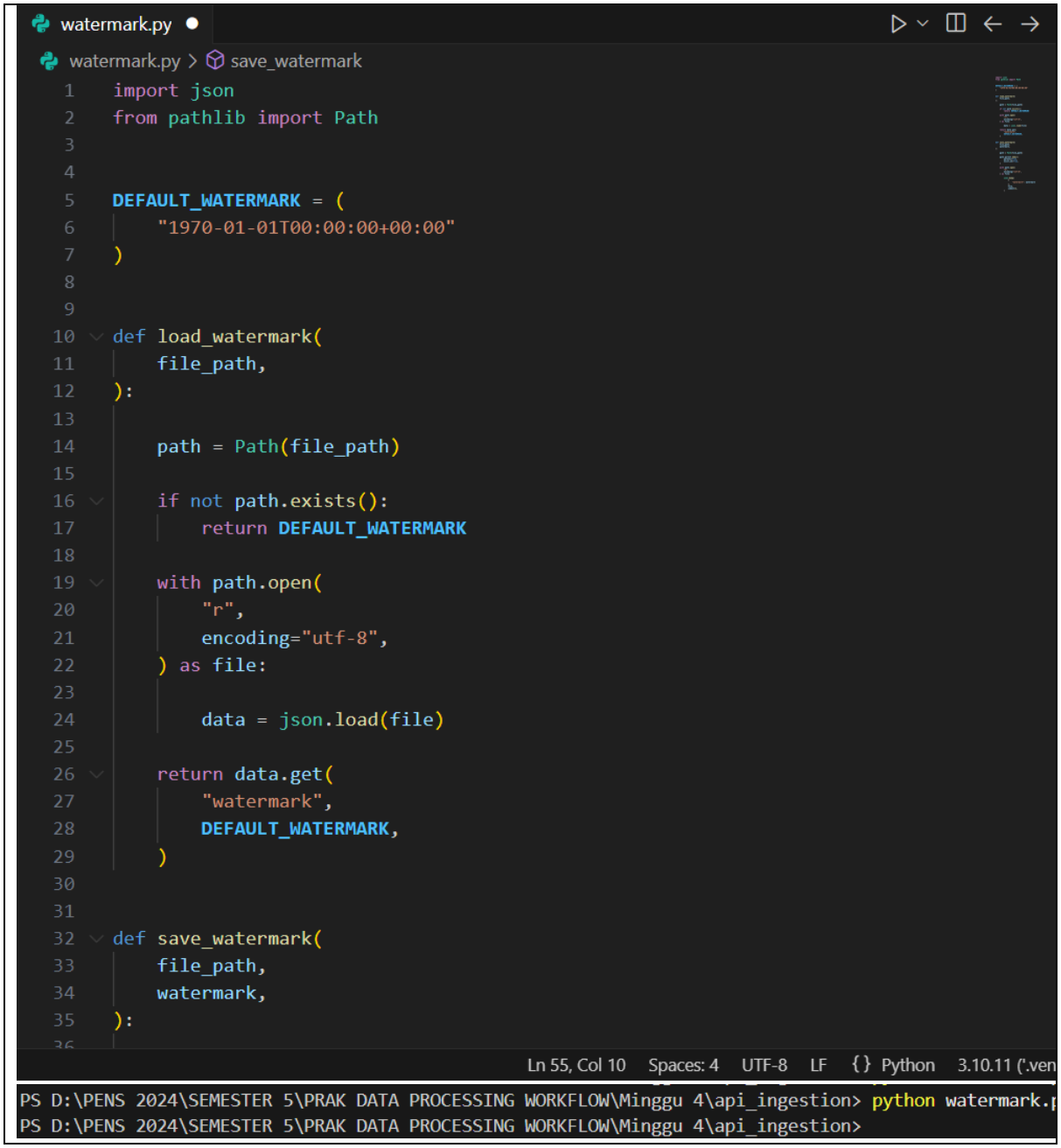
2026-09-01T12:00:00+00:00

Run berikutnya:

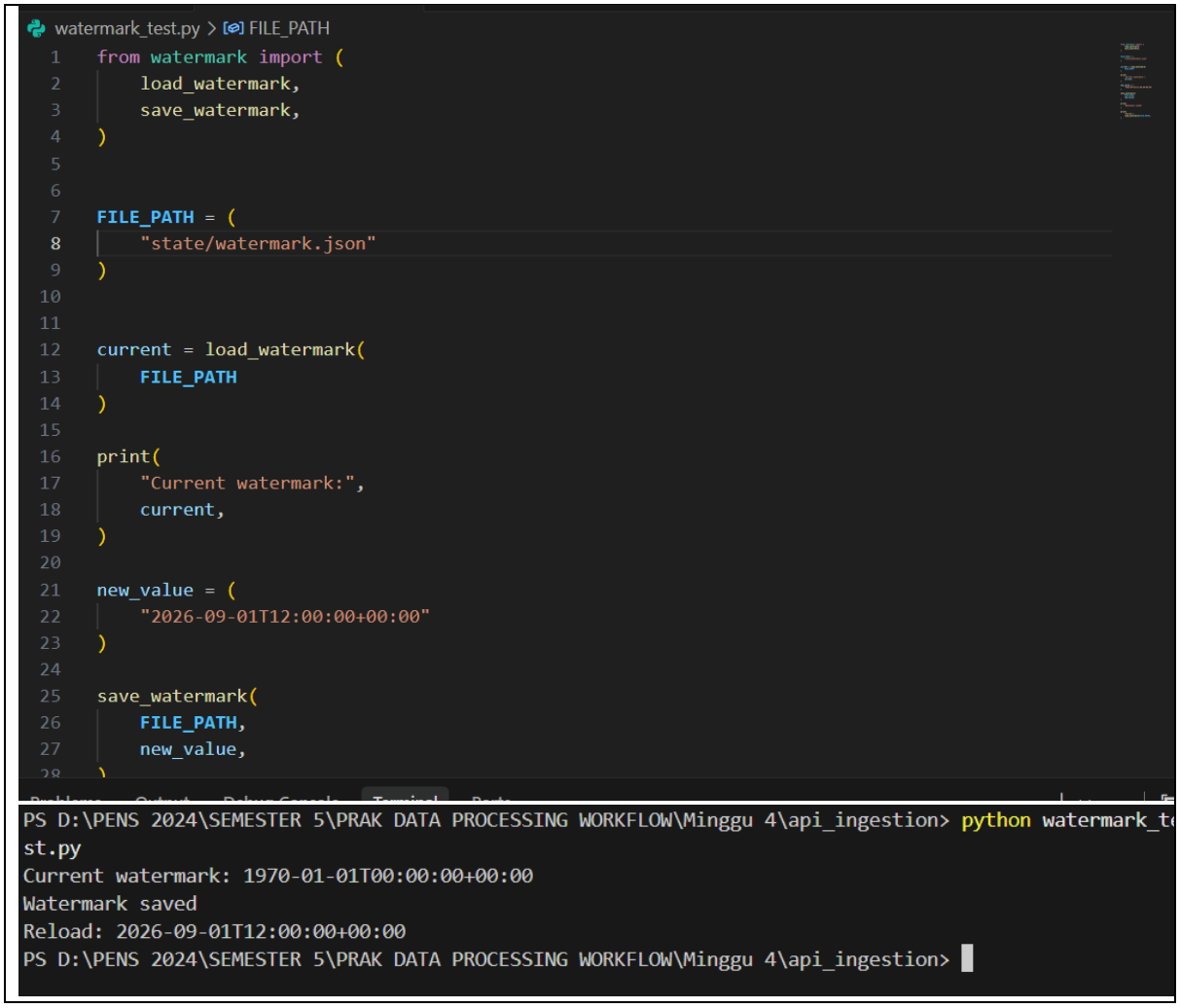
GET /transactions?

updated_since=2026-09-01T12:00:00+00:00

12.3 Buat watermark.py



12.4 Uji Watermark



12.5 Menggunakan Watermark pada API

watermark_api.py > ...

```python
import requests

from watermark import load_watermark


watermark = load_watermark(
    "state/watermark.json"
)

response = requests.get(
    (
        "http://127.0.0.1:8000/"
        "transactions"
    ),
    params={
        "page": 1,
        "limit": 10,
        "updated_since": watermark,
    },
    timeout=(3, 10),
)

response.raise_for_status()

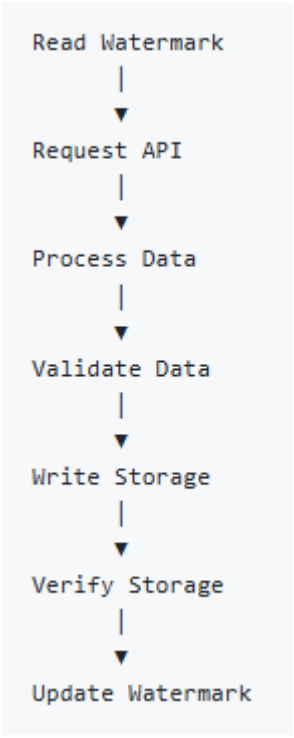
payload = response.json()

print(
    payload["data"]
)

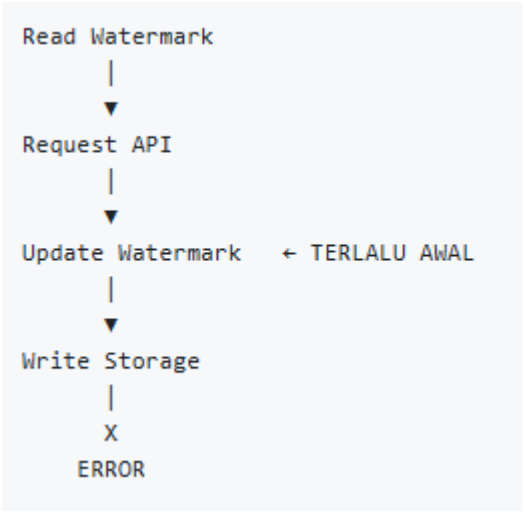
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python watermark_a
i.py
[{'id': 25, 'customer': 'customer_025', 'amount': 250000, 'updated_at': '2026-09-01T12:10:00+00:00'
, {'id': 26, 'customer': 'customer_026', 'amount': 260000, 'updated_at': '2026-09-01T12:20:00+00:00
}, {'id': 27, 'customer': 'customer_027', 'amount': 270000, 'updated_at': '2026-09-01T12:30:00+00:0
'}, {'id': 28, 'customer': 'customer_028', 'amount': 280000, 'updated_at': '2026-09-01T12:40:00+00:
0'}, {'id': 29, 'customer': 'customer_029', 'amount': 290000, 'updated_at': '2026-09-01T12:50:00+00
00'}, {'id': 30, 'customer': 'customer_030', 'amount': 300000, 'updated_at': '2026-09-01T13:00:00+0
:00'}, {'id': 31, 'customer': 'customer_031', 'amount': 310000, 'updated_at': '2026-09-01T13:10:00+
0:00'}, {'id': 32, 'customer': 'customer_032', 'amount': 320000, 'updated_at': '2026-09-01T13:20:00
00:00'}, {'id': 33, 'customer': 'customer_033', 'amount': 330000, 'updated_at': '2026-09-01T13:30:0
+00:00'}, {'id': 34, 'customer': 'customer_034', 'amount': 340000, 'updated_at': '2026-09-01T13:40:
0+00:00'}]
```

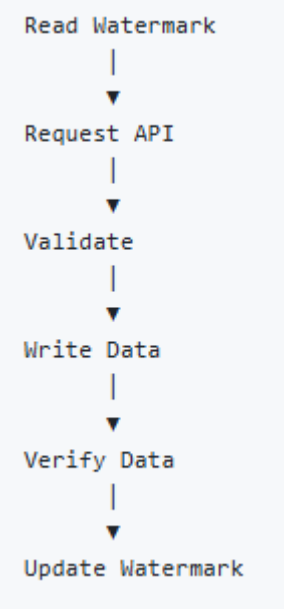
12.6 Siklus Watermark yang Benar



12.7 Pola Salah



12.8 Pola Aman



12.9 Menentukan Watermark Baru

Jika semua record berhasil:

```
new_watermark = max(
    record["updated_at"]
    for record in records
)
```

12.10 tugas

run_pipeline_watermark.py > fetch_and_process

```python
import os
from pathlib import Path
import pandas as pd
import requests
from watermark import load_watermark, save_watermark

FILE_PATH = "state/watermark.json"
API_URL = "http://127.0.0.1:8000/transactions"
PARQUET_DIR = Path("data/transactions")

def fetch_and_process(is_run_first=False):
    # 1. Read Watermark
    watermark_before = load_watermark(FILE_PATH)
    print(f"Watermark before: {watermark_before}")

    # 2. Request API dengan parameter updated_since
    params = {
        "page": 1,
        "limit": 100,
        "updated_since": watermark_before,
    }

    response = requests.get(API_URL, params=params, timeout=(3, 10))
    response.raise_for_status()

    payload = response.json()
    records = payload.get("data", [])

    records_fetched = len(records)
    print(f"Records fetched: {records_fetched}")
    Ctrl+L for Agent
    if records_fetched > 0:
        # 3. Process & Write Storage (Simpan ke Parquet)
        PARQUET_DIR.mkdir(parents=True, exist_ok=True)
        df = pd.DataFrame(records)

        # Simpan dengan nama unik (misal berdasarkan timestamp atau increment file)
        file_name = f"transactions_{'initial' if is_run_first else 'incremental'}.parquet"
        file_dest = PARQUET_DIR / file_name
        df.to_parquet(file_dest, index=False)
        print(f"Data saved to {file_dest}")

        # 4. Menentukan Watermark Baru (Bagian 12.9)
        new_watermark = max(record["updated_at"] for record in records)

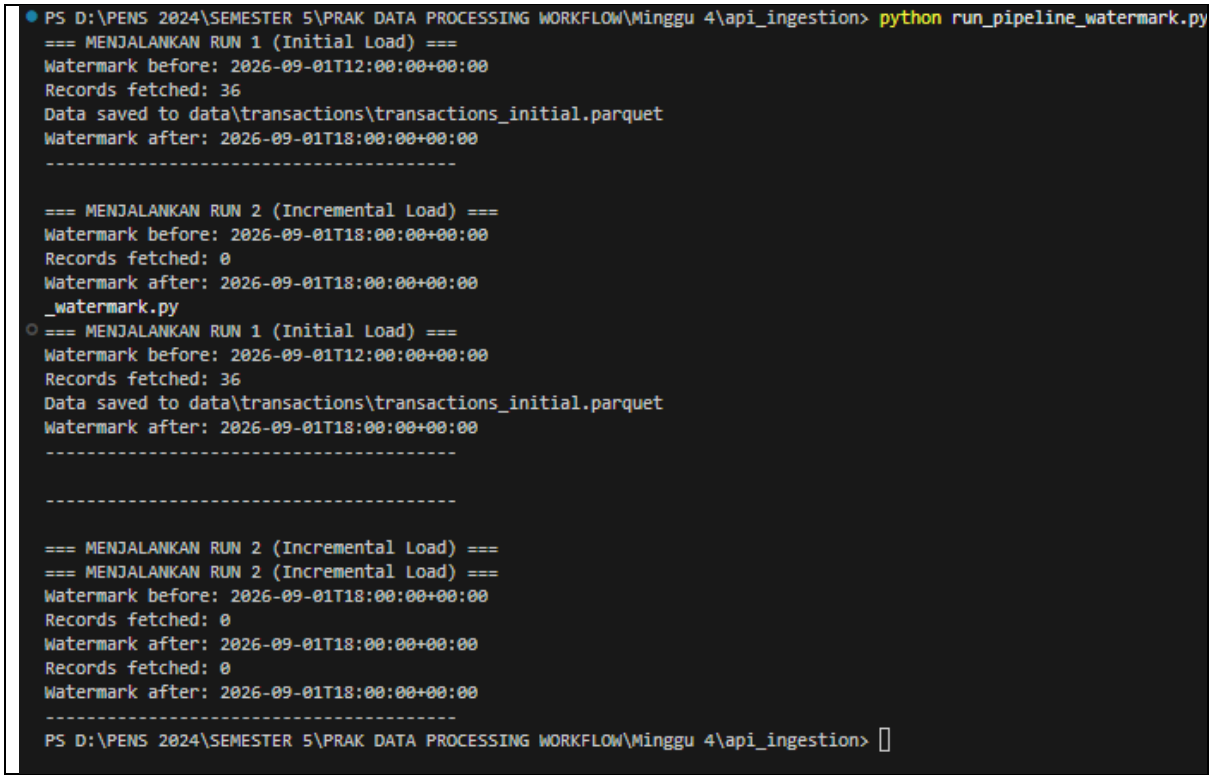
        # 5. Update Watermark (Dilakukan PALING AKHIR setelah sukses simpan)
        save_watermark(FILE_PATH, new_watermark)

    watermark_after = load_watermark(FILE_PATH)
    print(f"Watermark after: {watermark_after}")
    print("-" * 40)

if __name__ == "__main__":
    # Simulasi Run 1 (Hapus watermark.json dulu jika ingin simulasi dari awal)
    print("=== MENJALANKAN RUN 1 (Initial Load) ===")
    # Jika ingin reset total untuk Run 1, uncomment baris bawah:
    # if Path(FILE_PATH).exists(): Path(FILE_PATH).unlink()

    fetch_and_process(is_run_first=True)

    # Simulasi Run 2 (Incremental Load - jalankan lagi setelah ada data baru di API)
    print("\n=== MENJALANKAN RUN 2 (Incremental Load) ===")
    fetch_and_process(is_run_first=False)
```



ANALISIS:

Eksperimen ini memvalidasi konsep *incremental loading*, yakni metode efisiensi sistem dengan hanya mengambil kepingan data yang baru saja diperbarui. Teknik *watermark* berupa penanda waktu (timestamp) transaksi terakhir digunakan untuk menghindari duplikasi penarikan data lama yang berpotensi memboroskan sumber daya jaringan. Prinsip keamanan utamanya adalah *watermark* baru hanya boleh ditimpa oleh sistem setelah data pada sesi tersebut benar-benar tervalidasi dan berhasil disimpan.

13. PRAKTIKUM 7 — LOGGING DAN OBSERVABILITY

Tujuan

Menggantikan:

```
print("start")
print("page 1")
print("error")
```

dengan:

```
logging.info(...)
logging.warning(...)
logging.error(...)
```

13.1 Buat konfigurasi logging

Buat logging_setup.py atau letakkan fungsi berikut di main.py.

main.py > configure_logging

```python
import logging
from pathlib import Path


def configure_logging(
    log_file,
):

    path = Path(log_file)

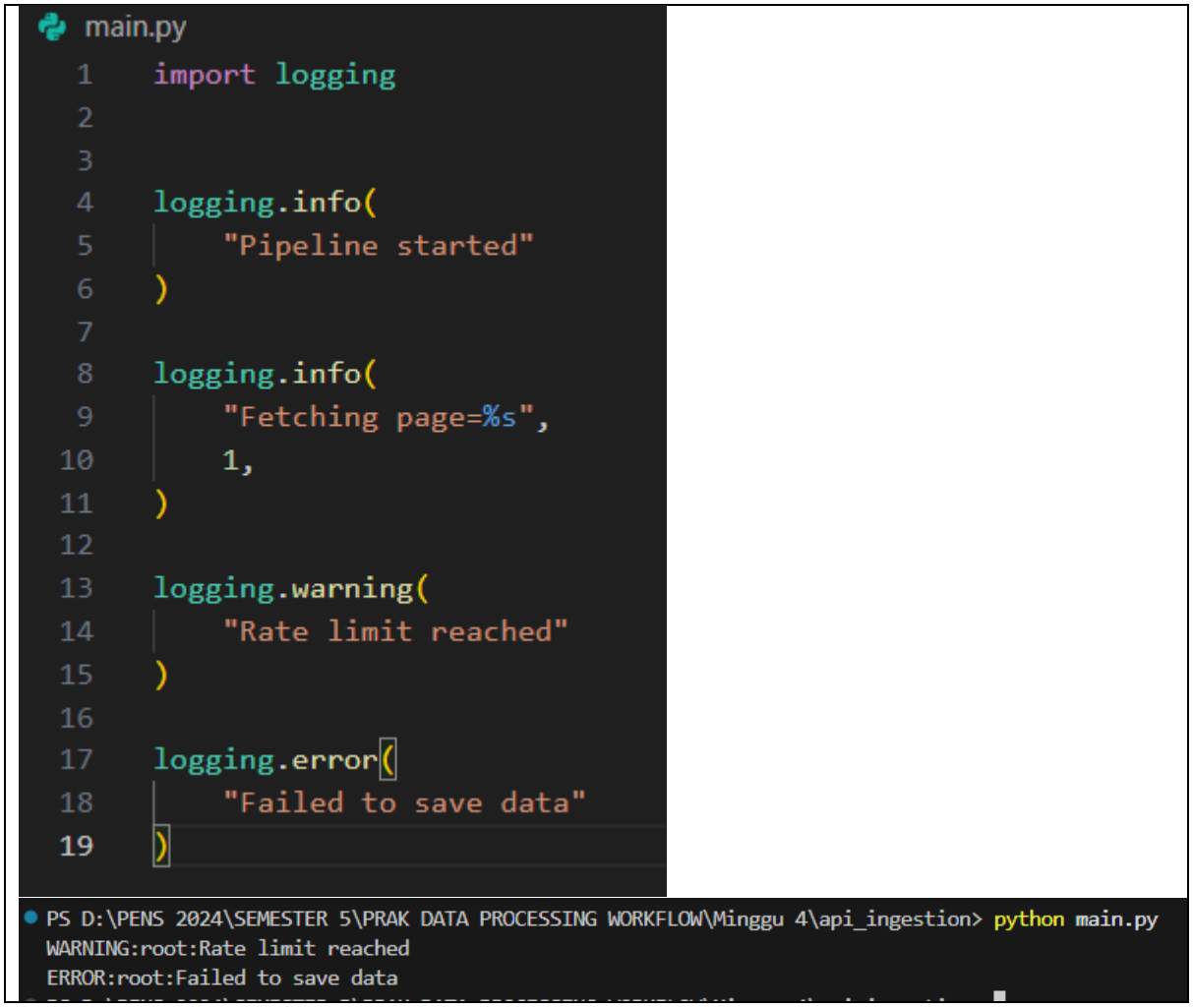
    path.parent.mkdir(
        parents=True,
        exist_ok=True,
    )

    logging.basicConfig(
        level=logging.INFO,
        format=(
            "%(asctime)s "
            "%(levelname)s "
            "%(name)s "
            "%(message)s"
        ),
        handlers=[
            logging.FileHandler(
                path,
                encoding="utf-8",
            ),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

13.2 Level Logging

Level	Kegunaan
DEBUG	Detail diagnosis
INFO	Aktivitas normal
WARNING	Kondisi tidak ideal
ERROR	Kegagalan
CRITICAL	Kegagalan serius

13.3 Contoh



13.4 Informasi yang Perlu Dicatat

Minimal:

- Pipeline started
- Watermark loaded
- Request page number
- Number of records
- HTTP status
- Retry attempt
- Wait time
- Output file
- Validation result
- Watermark updated
- Pipeline finished

13.5 Observability

Pertanyaan yang harus dapat dijawab dari log:

- Kapan pipeline berjalan?
- Berapa halaman yang diproses?
- Berapa record yang diambil?
- Apakah terjadi retry?
- Berapa lama backoff?
- File apa yang dihasilkan?
- Apakah penyimpanan berhasil?
- Watermark berubah menjadi apa?

13.6 Latihan

main.py > ...

```python
import logging
from pathlib import Path
import requests
import pandas as pd

# Import modul setup logging yang baru saja dibuat
from logging_setup import configure_logging
from watermark import load_watermark, save_watermark

# Inisialisasi logger untuk main pipeline
logger = logging.getLogger("MainPipeline")

FILE_PATH = "watermark.json" # atau sesuaikan path foldernya
API_URL = "http://127.0.0.1:8000/transactions"
PARQUET_DIR = Path("data/transactions")

def main():
    # 1. Konfigurasi Logging (menyimpan ke logs/ingestion.log dan tampil di console)
    configure_logging("logs/ingestion.log")

    logger.info("Pipeline started")

    try:
        # 2. Load Watermark
        watermark = load_watermark(FILE_PATH)
        logger.info("Watermark loaded: %s", watermark)

        # 3. Request API dengan parameter watermark
        logger.info("Request page number: 1")
        params = {
            "page": 1,
            "limit": 100,
            "updated_since": watermark,
        }

        logger.info("Requesting URL: %s with params: %s", API_URL, params)
        response = requests.get(API_URL, params=params, timeout=(3, 10))

        logger.info("HTTP status received: %s", response.status_code)
        response.raise_for_status()

        payload = response.json()
        records = payload.get("data", [])
```

logging_setup.py > configure_logging

```python
import logging
from pathlib import Path

def configure_logging(log_file="logs/ingestion.log"):
    path = Path(log_file)
    path.parent.mkdir(
        parents=True,
        exist_ok=True,
    )

    logging.basicConfig(
        level=logging.INFO,
        format=(
            "%(asctime)s "
            "%(levelname)s "
            "%(name)s "
            "%(message)s"
        ),
        handlers=[
            logging.FileHandler(
                path,
                encoding="utf-8",
            ),
            logging.StreamHandler(),
        ],
        force=True,
    )
```

logs > ingestion.log

```
2026-09-24 23:32:20,286 INFO MainPipeline Pipeline started
2026-09-24 23:32:20,312 INFO MainPipeline Watermark loaded: 1970-01-01T00:00:00+00:00
2026-09-24 23:32:20,313 INFO MainPipeline Request page number: 1
2026-09-24 23:32:20,314 INFO MainPipeline Requesting URL: http://127.0.0.1:8000/transactions with params: {'page': 1, 'limit':
2026-09-24 23:32:20,342 INFO MainPipeline HTTP status received: 200
2026-09-24 23:32:20,343 INFO MainPipeline Number of records fetched: 60
2026-09-24 23:32:20,344 INFO MainPipeline Validation result: Passed (60 records)
2026-09-24 23:32:20,401 INFO MainPipeline Output file successfully generated: data\transactions\transactions_incremental.parque
2026-09-24 23:32:20,403 INFO MainPipeline Watermark updated to: 2026-09-01T18:00:00+00:00
2026-09-24 23:32:20,404 INFO MainPipeline Pipeline finished successfully
```

ANALISIS:

Tahapan ini melakukan transisi dari perintah mencetak standar (print) menuju pemanfaatan pustaka logging bawaan Python untuk pemantauan sistem (*observability*). Pendekatan ini bertujuan agar rekam jejak berjalannya program (*pipeline*) menjadi lebih terstruktur, memiliki konteks waktu yang persis, dan mudah dilacak untuk keperluan pemecahan masalah (*debugging*). Melalui pembagian level

klasifikasi *log* yang jelas (seperti *INFO* atau *ERROR*), analisis riwayat eksekusi dan pelacakan kegagalan dapat dilakukan secara komprehensif.

14. PRAKTIKUM 8 — VALIDASI DATA

Tujuan

Jangan langsung menyimpan data tanpa memeriksa struktur.

14.1 Buat validator.py

validator.py > validate_records

```python
REQUIRED_FIELDS = {
    "id",
    "customer",
    "amount",
    "updated_at",
}


def validate_record(
    record,
):

    missing_fields = (
        REQUIRED_FIELDS
        - set(record.keys())
    )

    if missing_fields:

        raise ValueError(
            "Missing fields: "
            f"{sorted(missing_fields)}"
        )

    if not isinstance(
        record["id"],
        int,
    ):
        raise ValueError(
            "id must be int"
        )

    if not isinstance(
        record["customer"],
        str,
    ):
        raise ValueError(
            "customer must be str"
        )

    if not isinstance(
        record["amount"],
        (int, float),
    ):
        raise ValueError(
            "amount must be numeric"
        )


def validate_records(
    records,
):

    if not isinstance(
        records,
        list,
    ):
        raise ValueError(
            "records must be a list"
        )

    for record in records:
        validate_record(record)

```

14.2 Uji Validasi

```
    from validator import validate_records


records = [
    {
        "id": 1,
        "customer": "customer_001",
        "amount": 10000,
        "updated_at": (
            "2026-09-01T08:10:00+00:00"
        ),
    }
]


validate_records(records)

print(
    "Validation passed"
)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python validator.py
Validation passed
```

14.3 Uji Error

Before:

```
71
72  records = [
73      {
74          "id": 1,
75          "customer": "customer_001",
76          "amount": 10000,
77          "updated_at": (
78              "2026-09-01T08:10:00+00:00"
79          ),
80      }
81  ]
82
83
84  validate_records(records)
85
86  print(
87      "Validation passed"
88  )
```

After:

```
records = [
    {
        "id": 1,
        "customer": "customer_001",
        "updated_at": (
            "2026-09-01T08:10:00+00:00"
        ),
    }
]


validate_records(records)

print(
    "Validation passed"
)
```

Result:

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python validator.py
Traceback (most recent call last):
  File "D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion\validator.py", line 83, in <module>
    validate_records(records)
  File "D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion\validator.py", line 63, in validate_records
    validate_record(record)
  File "D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion\validator.py", line 20, in validate_record
    raise ValueError(
ValueError: Missing fields: ['amount']
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

14.4 Validasi Duplikasi

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python validator.py
Validation passed
```

ANALISIS:

Praktikum ini merancang mekanisme pengecekan kelengkapan kolom wajib dan kesesuaian tipe data sebelum program menuliskan rekam data ke dalam media penyimpanan. Proses penyaringan ketat ini dilakukan dengan tujuan menjamin bahwa data yang diproses benar-benar bersih serta bebas dari anomali, seperti duplikasi nilai ID. Standardisasi di tahap awal ini sangat vital untuk mencegah masuknya data kotor yang berisiko merusak integritas hasil analitik di downstream (garbage in, garbage out).

15. PRAKTIKUM 9 — APACHE PARQUET

Tujuan

Menyimpan hasil ingestion dalam format kolumnar.

15.1 Mengapa Tidak Selalu CSV?

```
id,customer,amount
```

```
1,A,100
2,B,200
```

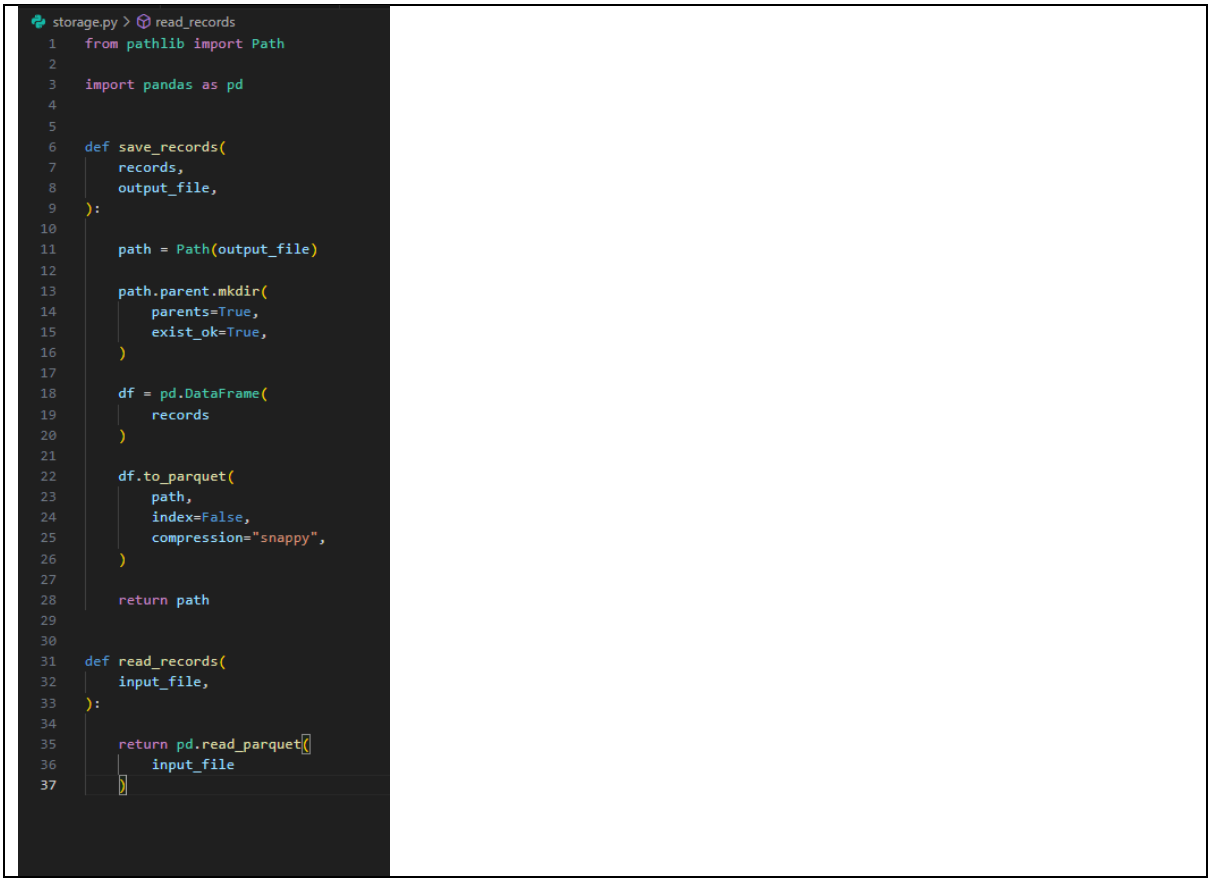
Keunggulan:

- Sederhana.
- Mudah dibaca.
- Namun untuk analitik skala lebih besar, Parquet menawarkan:

Penyimpanan kolumnar.

- Compression.
- Metadata.
- Integrasi dengan ekosistem analitik.

15.2 Buat storage.py



15.3 Menulis Parquet

```python
from storage import save_records


records = [
    {
        "id": 1,
        "customer": "Andi",
        "amount": 10000,
        "updated_at": (
            "2026-09-01T08:00:00+00:00"
        ),
    },
    {
        "id": 2,
        "customer": "Budi",
        "amount": 20000,
        "updated_at": (
            "2026-09-01T09:00:00+00:00"
        ),
    },
]


output_file = (
    "data/transactions.parquet"
)

save_records(
    records,
    output_file,
)

print(
    "Parquet saved"
)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python storage.py
Parquet saved
```

15.4 Membaca Kembali

```python
from storage import read_records


df = read_records(
    "data/transactions.parquet"
)

print(df)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python storage.py
Parquet saved
   id customer  amount                updated_at
0   1     Andi   10000  2026-09-01T08:00:00+00:00
1   2     Budi   20000  2026-09-01T09:00:00+00:00
Parquet saved
   id customer  amount                updated_at
0   1     Andi   10000  2026-09-01T08:00:00+00:00
1   2     Budi   20000  2026-09-01T09:00:00+00:00
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

15.5 Validasi Output

```python
import pandas as pd


input_records = [
    {
        "id": 1,
        "customer": "Andi",
        "amount": 10000,
        "updated_at": "2026-09-01",
    }
]


df = pd.DataFrame(
    input_records
)

output_file = (
    "data/validation.parquet"
)

df.to_parquet(
    output_file,
    index=False,
)

result = pd.read_parquet(
    output_file
)

assert len(result) == len(df)

assert list(result.columns) == list(
    df.columns
)

print(
    "Parquet validation passed"
)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python storage.py
Parquet saved
   id customer  amount                 updated_at
0   1     Andi   10000  2026-09-01T08:00:00+00:00
1   2     Budi   20000  2026-09-01T09:00:00+00:00
Parquet validation passed
Parquet saved
   id customer  amount                 updated_at
0   1     Andi   10000  2026-09-01T08:00:00+00:00
1   2     Budi   20000  2026-09-01T09:00:00+00:00
Parquet validation passed
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

15.6 Eksperimen Compression

test_compression.py > ...

```python
from pathlib import Path
import pandas as pd
from storage import save_records

# 1. Generate dataset yang agak besar (misal 100.000 baris) agar kompresi terasa
print("Membuat dataset dummy...")
records = [
    {
        "id": i,
        "customer": f"customer_{i % 1000}",
        "amount": i * 150,
        "updated_at": "2026-09-01T08:00:00+00:00"
    }
    for i in range(100000)
]

# Pastikan direktori data ada
Path("data").mkdir(parents=True, exist_ok=True)

# 2. Uji dengan kompresi SNAPPY
file_snappy = "data/transactions_snappy.parquet"
print(f"Menyimpan dengan kompresi 'snappy' ke {file_snappy}...")
# Memodifikasi pemanggilan dengan parameter compression
path_snappy = Path(file_snappy)
path_snappy.parent.mkdir(parents=True, exist_ok=True)
pd.DataFrame(records).to_parquet(path_snappy, index=False, compression="snappy")

# 3. Uji dengan kompresi GZIP
file_gzip = "data/transactions_gzip.parquet"
print(f"Menyimpan dengan kompresi 'gzip' ke {file_gzip}...")
path_gzip = Path(file_gzip)
path_gzip.parent.mkdir(parents=True, exist_ok=True)
pd.DataFrame(records).to_parquet(path_gzip, index=False, compression="gzip")

# 4. Tampilkan perbandingan ukuran file (dalam ukuran KB / MB)
size_snappy = Path(file_snappy).stat().st_size / 1024
size_gzip = Path(file_gzip).stat().st_size / 1024

print("\n" + "="*30)
print("HASIL EKSPERIMEN KOMPRESI")
print("="*30)
print(f"Ukuran file Snappy : {size_snappy:.2f} KB")
print(f"Ukuran file Gzip   : {size_gzip:.2f} KB")
print("="*30)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python test_compression.py
Parquet saved
   id customer  amount                updated_at
0   1     Andi   10000  2026-09-01T08:00:00+00:00
1   2     Budi   20000  2026-09-01T09:00:00+00:00
Parquet validation passed
Membuat dataset dummy...
Menyimpan dengan kompresi 'snappy' ke data/transactions_snappy.parquet...
Menyimpan dengan kompresi 'gzip' ke data/transactions_gzip.parquet...

==============================
HASIL EKSPERIMEN KOMPRESI
==============================
Ukuran file Snappy : 1258.03 KB
Ukuran file Gzip   : 735.07 KB
==============================
```

ANALISIS:

Percobaan ini melakukan evaluasi format penyimpanan, menyoroti efisiensi format data kolumnar Apache Parquet apabila disandingkan dengan format teks baris konvensional seperti CSV. Pengujian algoritma kompresi bawaan (Snappy dan Gzip) diimplementasikan dengan tujuan meminimalkan ruang penyimpanan berkas secara signifikan tanpa mengorbankan kecepatan baca/tulis datanya.

Percobaan ini membuktikan adanya trade-off (pertukaran prioritas) dalam memilih format penyimpanan data. Algoritma Gzip sangat unggul dalam memampatkan ukuran file (menghemat ruang hingga lebih dari 40%), sehingga sangat ideal jika fokus utama Anda adalah meminimalkan biaya storage (seperti arsip data lama). Di sisi lain, meskipun algoritma Snappy menghasilkan ukuran file yang lebih besar, algoritma ini menjadi standar default di ekosistem Big Data (seperti Apache Spark) karena kecepatan compress/decompress-nya yang sangat tinggi, memungkinkan proses analitik berjalan jauh lebih cepat.

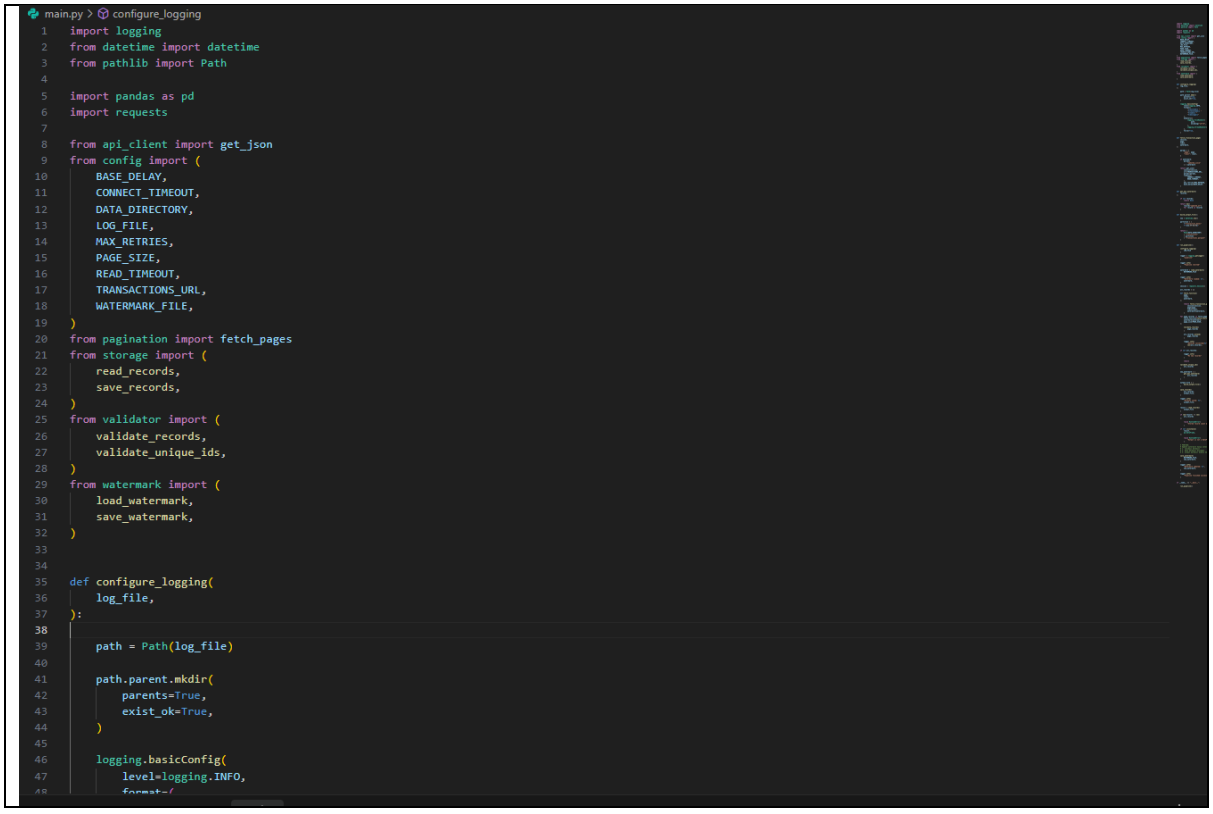
Hasil pengujian menunjukkan bahwa spesifikasi Parquet sangat ideal untuk diaplikasikan pada industri yang membutuhkan analitik Big Data tingkat lanjut.

16. PRAKTIKUM 10 — MEMBANGUN PIPELINE END-TO-END

Ini adalah inti praktikum.

```
Configuration
      |
      ▼
Logging
      |
      ▼
Read Watermark
      |
      ▼
Fetch API
      |
      ▼
Retry / Backoff / Jitter
      |
      ▼
Pagination
      |
      ▼
Validation
      |
      ▼
Collect Batch
      |
      ▼
Write Parquet
      |
      ▼
Verify Output
      |
      ▼
Update Watermark
```

16.1 Buat main.py



16.2 perbaiki config.py dan api_client.py

config.py > LOG_FILE

```python
API_BASE_URL = (
    "http://127.0.0.1:8000"
)

TRANSACTIONS_URL = (
    f"{API_BASE_URL}/transactions"
)

PAGE_SIZE = 10

CONNECT_TIMEOUT = 3
READ_TIMEOUT = 10

MAX_RETRIES = 5
BASE_DELAY = 1

WATERMARK_FILE = (
    "state/watermark.json"
)

DATA_DIRECTORY = "data"

LOG_FILE = (
    "logs/ingestion.log"
)
```

api_client.py > get_json

```python
"""
api_client.py — Fungsi HTTP request dengan retry, exponential backoff, jitter,
                dan penanganan rate limit (429).
"""

import time
import random
import logging
import requests

import config

logger = logging.getLogger(__name__)

import time
import requests
import logging

logger = logging.getLogger(__name__)

def get_json(session, url, params, timeout, max_retries, base_delay):
    for attempt in range(1, max_retries + 1):
        try:
            # Perbaikan: gunakan session.get dan parameter timeout bawaan fungsi
            response = session.get(url, params=params, timeout=timeout)

            response.raise_for_status()
            return response.json()

        except requests.exceptions.RequestException as e:
            logger.warning("Attempt %s failed: %s", attempt, e)
            if attempt == max_retries:
                logger.error("Max retries reached. Failed to fetch %s", url)
                raise

            sleep_time = base_delay * (2 ** (attempt - 1))
            logger.info("Waiting %s seconds before retry...", sleep_time)
            time.sleep(sleep_time)

    for attempt in range(1, config.MAX_RETRIES + 1):
        logger.debug(f"Request attempt {attempt}/{config.MAX_RETRIES} | url={url} | params={params}")

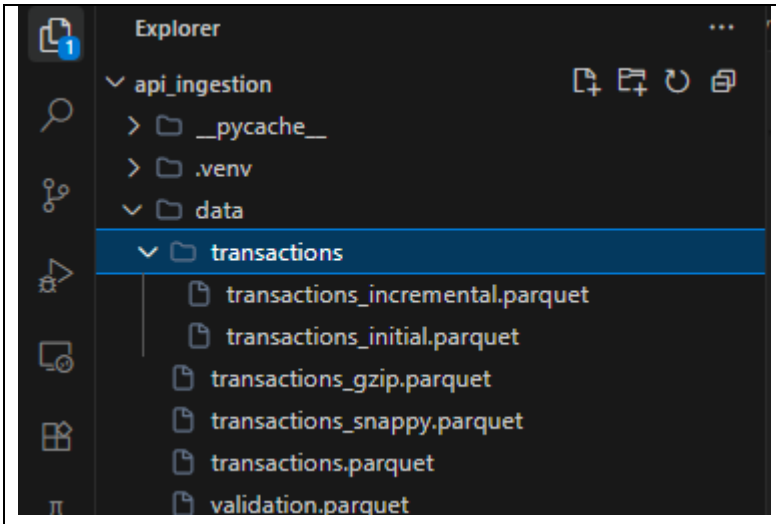
        try:
            response = requests.get(url, params=params, timeout=config.TIMEOUT)

            # ── 429: Rate Limit ──────────────────────────────
            if response.status_code == 429:
                retry_after = int(response.headers.get("Retry-After", config.DEFAULT_RETRY_AFTER))
```

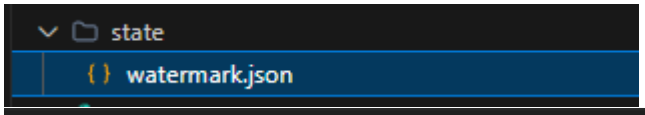
16.3 jalankan pipeline

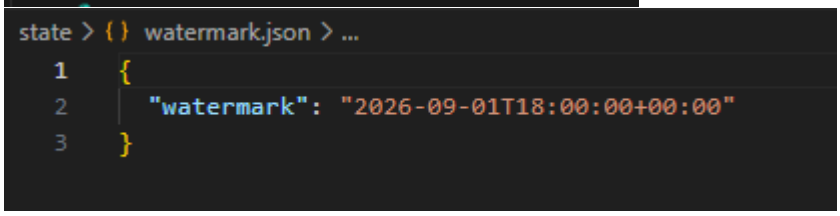
```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python main.py
2026-09-25 06:04:06,927 INFO pipeline Pipeline started
2026-09-25 06:04:06,927 INFO pipeline Watermark loaded: 2026-09-01T18:00:00+00:00
Fetching page 1 with watermark=2026-09-01T18:00:00+00:00
No more data (empty list)
2026-09-25 06:04:06,935 INFO pipeline No new records
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

16.4 periksa output

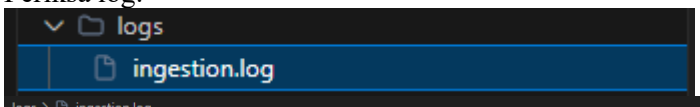


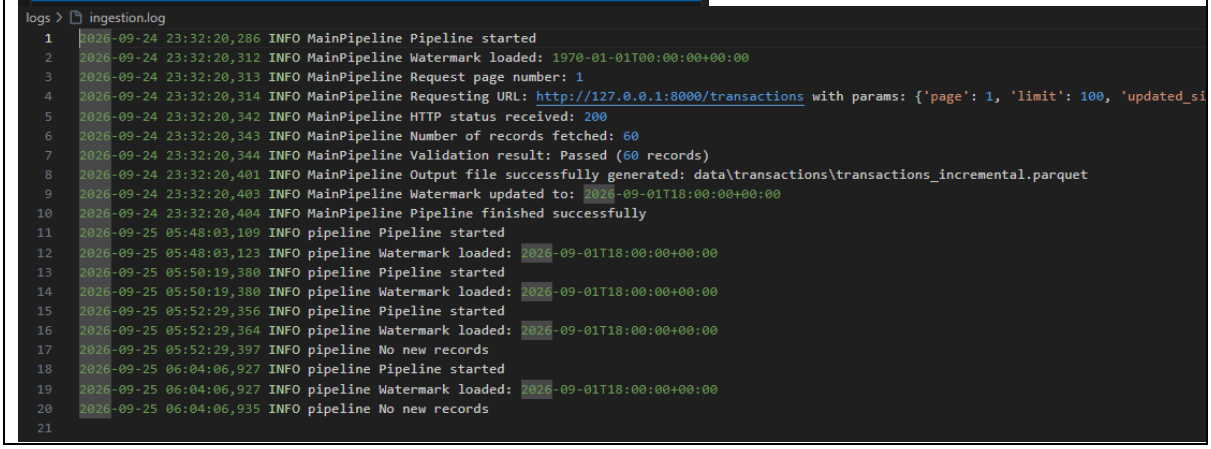
Periksa wateermark:





Periksa log:





16.5 Uji Run Kedua

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python main.py
2026-09-25 06:10:15,417 INFO pipeline Pipeline started
2026-09-25 06:10:15,418 INFO pipeline Watermark loaded: 2026-09-01T18:00:00+00:00
Fetching page 1 with watermark=2026-09-01T18:00:00+00:00
No more data (empty list)
2026-09-25 06:10:15,444 INFO pipeline No new records
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion>
```

ANALISIS:

Percobaan puncak ini mengintegrasikan seluruh modul independen yang telah dibuat menjadi satu arsitektur *pipeline* data utuh yang mampu beroperasi secara terorkestrasi. Alur kerja mulai dari inisiasi konfigurasi, penanganan *retry*, iterasi *pagination* API, hingga proses akhir berupa validasi dan penulisan berkas Parquet berhasil dieksekusi secara berurutan. Hasil evaluasi pada uji coba langkah kedua, di mana sistem secara presisi mendeteksi ketiadaan data baru ("No new records"), membuktikan bahwa logika operasional *incremental loading* telah berfungsi dengan sempurna.

17. PRAKTIKUM 11 — STRATEGI PENYIMPANAN BATCH DAN PARTITIONING

Masalah

Jangan selalu:

```
data/
└── transactions.parquet
```

untuk seluruh data.

Alternatif:

```
data/
└── transactions/
    ├── transaction_date=2026-09-01/
    │   └── transactions.parquet
    ├── transaction_date=2026-09-02/
    │   └── transactions.parquet
    └── transaction_date=2026-09-03/
        └── transactions.parquet
```

17.1 Partition Berdasarkan Tanggal



17.2 Mengapa Partitioning Berguna?

Jika analisis membutuhkan:

Data tanggal 2026-09-01

sistem tidak harus selalu membaca seluruh koleksi.

Namun strategi partisi harus mempertimbangkan:

- Query pattern.
- Jumlah file.
- Volume data.
- Ukuran batch.
- Kebutuhan downstream.

ANALISIS:

Percobaan ini mendemonstrasikan transisi krusial dari arsitektur penyimpanan data flat (semua data ditumpuk dalam satu file) menuju arsitektur berpartisi dinamis yang siap untuk skala produksi (production-grade). Berdasarkan fungsi build_output_file(), berikut adalah analisis sistemnya:

- Penerapan Hive-Style Partitioning:

Sistem secara dinamis membuat direktori penyimpanan menggunakan format key=value, yaitu transaction_date=YYYY-MM-DD. Format ini merupakan standar industri yang secara otomatis dapat dikenali oleh mesin Big Data (seperti Apache Spark, Amazon Athena, atau Google BigQuery) sebagai kolom partisi (partition key) tanpa harus mengekstrak nilai tersebut dari dalam file Parquet itu sendiri.

- Pencegahan Anomali Data Swamp (Skalabilitas):

Dengan memecah data per hari berdasarkan waktu eksekusi (datetime.now()), ukuran masing-masing file Parquet akan tetap kecil dan optimal. Hal ini mencegah terjadinya bottleneck I/O (Input/Output)

yang sering terjadi jika sistem terus-menerus mencoba menulis ulang atau menambahkan (append) data ke satu file raksasa secara berulang-ulang.

- Optimasi Kueri Downstream (Data Skipping):

Strategi partisi ini sangat menguntungkan tim analis di tahap downstream. Jika seorang analis hanya membutuhkan data transaksi untuk tanggal 1 hingga 3 September, mesin kueri hanya akan membaca direktori yang relevan dan mengabaikan (skip) ratusan direktori tanggal lainnya. Ini menghemat penggunaan memori, waktu proses komputasi, dan biaya kueri cloud secara dramatis.

- Keandalan dan Isolasi Data (Idempotency):

Desain ini memastikan keamanan dalam pemeliharaan data. Jika terjadi kegagalan sistem atau data kotor pada satu tanggal tertentu, Anda atau tim Data Engineer hanya perlu menghapus dan menjalankan ulang (backfill) direktori untuk tanggal tersebut saja, tanpa ada risiko merusak riwayat data transaksi di hari-hari sebelumnya.

Kesimpulan:

Tahapan pamungkas ini menerapkan strategi penyimpanan berpartisi (Hive-style partitioning) yang memecah distribusi keluaran data secara dinamis ke dalam hierarki direktori berdasarkan parameter tanggal eksekusi. Pemisahan kelompok direktori ini dikembangkan dengan tujuan mengakselerasi proses kueri downstream, sehingga mesin analitik hanya perlu membaca sub-folder yang relevan tanpa harus memindai keseluruhan penyimpanan data historis (data skipping). Implementasi arsitektur fisik ini menjadi fondasi yang krusial guna menjaga performa dan efisiensi biaya pada skalabilitas data warehouse.

Tugas A — Pagination



Praktik B — Retry

Fungsi ini menggabungkan penanganan Timeout, Exponential Backoff, dan Jitter (pengacakan waktu tunggu agar tidak terjadi antrean request serentak), serta memfilter hanya status server error (5xx) yang diulang.

praktik_B.py > get_with_retry

date

```python
import requests
import time
import random

def get_with_retry(url):
    max_retries = 3
    timeout = (3, 10) # 3 detik connect, 10 detik read
    base_delay = 1
    retryable_statuses = {500, 502, 503, 504}

    for attempt in range(1, max_retries + 1):
        try:
            response = requests.get(url, timeout=timeout)
            response.raise_for_status() # Cek apakah ada error HTTP
            return response.json()

        except requests.exceptions.HTTPError as e:
            status = e.response.status_code

            # Jika error bersifat client-side (seperti 400, 401, 403, 404), langsung hentikan
            if status not in retryable_statuses:
                raise

            # Jika mencapai batas maksimum percobaan, lempar errornya
            if attempt == max_retries:
                raise

        except requests.exceptions.RequestException:
            # Menangkap error jaringan seperti ConnectionError atau Timeout
            if attempt == max_retries:
                raise

        # Kalkulasi Exponential Backoff + Jitter
        sleep_time = (base_delay * (2 ** (attempt - 1))) + random.uniform(0, 1)
        time.sleep(sleep_time)
```

Praktik C — Watermark

Skrip ini memastikan state (titik rekam terakhir) dari pipeline tersimpan rapi dalam format JSON, meng-handle kondisi jika sistem dijalankan untuk pertama kalinya (initial load), dan secara otomatis membuat struktur folder yang dibutuhkan.

praktik_C_watermark.py

```python
import json
from pathlib import Path

DEFAULT_WATERMARK = "1970-01-01T00:00:00+00:00"

def load_watermark(file_path="state/watermark.json"):
    path = Path(file_path)

    # Syarat: Jika file belum ada, gunakan default
    if not path.exists():
        return DEFAULT_WATERMARK

    with path.open("r", encoding="utf-8") as file:
        data = json.load(file)

    return data.get("watermark", DEFAULT_WATERMARK)

def save_watermark(watermark, file_path="state/watermark.json"):
    path = Path(file_path)

    # Syarat: Folder dibuat otomatis
    path.parent.mkdir(parents=True, exist_ok=True)

    with path.open("w", encoding="utf-8") as file:
        # Syarat: Format JSON yang spesifik
        json.dump({"watermark": watermark}, file, indent=2)
```

Praktik D — Parquet

praktik_D_parquet.py > ...

```python
import pandas as pd


data = [
    {
        "id": 1,
        "name": "Andi",
        "amount": 10000,
    },
    {
        "id": 2,
        "name": "Budi",
        "amount": 20000,
    },
]

df = pd.DataFrame(
    data
)

df.to_parquet(
    "data.parquet",
    index=False,
)

result = pd.read_parquet(
    "data.parquet"
)

print(result)
```

```
PS D:\PENS 2024\SEMESTER 5\PRAK DATA PROCESSING WORKFLOW\Minggu 4\api_ingestion> python praktik_D_parquet.py
   id  name  amount
0   1  Andi   10000
1   2  Budi   20000
```

19. CASE STUDY — TRANSACTION API

Kasus

Sebuah perusahaan memiliki API:

GET /transactions

Response:

```
{
 "data": [
   {
    "id": 1001,
```

```
    "amount": 25000,
    "updated_at": "2026-09-01T10:00:00+00:00"
   }
 ],
 "next": "..."
}
```


Kondisi:

- API memiliki jutaan transaksi.
- API menggunakan pagination.
- API memiliki rate limit.
- Server kadang mengalami 500.
- Data dapat berubah.
- Pipeline dijalankan berkala.s
- Data harus disimpan sebagai Parquet.

20.1 Pertanyaan Analisis

- Mengapa tidak cukup menggunakan satu requests.get()?

API memiliki jutaan transaksi. Meminta semuanya dalam satu requests.get() akan memicu timeout dari server, membebani bandwidth, dan menyebabkan Out of Memory (OOM) pada sistem karena mencoba memuat jutaan baris data sekaligus ke dalam RAM.

- Pagination apa yang digunakan?

Berdasarkan response {"next": "..."}, API ini menggunakan jenis pagination Cursor-based Pagination atau Link-based Pagination. Client harus menggunakan kursor atau URL dari field next tersebut untuk memanggil halaman berikutnya.

- Apa kondisi berhentinya?

Pipeline berhenti mengambil data ketika nilai dari field next adalah null/kosong, atau ketika array "data": [] yang dikembalikan sudah tidak memiliki elemen.

- Error mana yang dapat di-retry?

Error yang wajib dan aman untuk diulang adalah kendala jaringan (timeout, connection error) dan kegagalan internal server (HTTP 500, 502, 503, 504). Kegagalan karena client (seperti 400 Bad Request atau 404 Not Found) tidak boleh di-retry.

- Bagaimana menangani 429?

Menghentikan exception 429 dan periksa apakah API mengirimkan header Retry-After. Jika ada, jeda eksekusi (sleep) sesuai instruksi server. Jika tidak ada, terapkan Exponential Backoff yang dikombinasikan dengan Jitter (pengacakan jeda) sebelum mengulang request.

- Bagaimana menentukan watermark?

Diperoleh dengan mencari nilai waktu paling tinggi/maksimum (max()) dari field updated_at pada seluruh record transaksi yang sukses ditarik di sesi tersebut.

- Kapan watermark diperbarui?

Wajib dilakukan paling akhir—hanya setelah data ditarik seluruhnya, lolos tahap validasi, selesai ditulis dengan sempurna ke format Parquet, dan lolos verifikasi baca ulang.

- Bagaimana mencegah data duplikat?

Filter dilakukan di awal dengan menyematkan parameter incremental (seperti ?updated_since=...) menggunakan nilai watermark terakhir. Karena studi kasus menyebutkan "Data dapat berubah", penanganan lanjutan diperlukan di downstream (saat diolah di data warehouse atau menggunakan mesin seperti Apache Spark/Delta Lake) dengan melakukan deduplikasi (upsert) berdasarkan id dan mengambil updated_at paling mutakhir.

- Bagaimana menyimpan output?

Data dikonversi menjadi DataFrame dan disimpan dalam format Apache Parquet. Sistem harus memecah penyimpanan ini dengan teknik Partitioning berdasar tanggal (misalnya di folder transaction_date=2026-09-25/) agar tidak menumpuk dalam satu file raksasa (Data Swamp).

- Informasi apa yang perlu dicatat pada log?

Minimal mencatat waktu pipeline mulai/berakhir, nilai watermark awal yang dimuat, parameter halaman/URL yang sedang di-request, status HTTP, riwayat retry (nomor percobaan dan lama backoff), jumlah record yang ditarik, hasil status validasi, path direktori tempat file Parquet disimpan, dan nilai watermark baru.